

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INFORMATICA



Tesi di laurea di I livello in
Informatica

Human-Computer Interaction & Music: Sintetizzatore Sonoro In Unity

Relatore:

Rocco Zaccagnino

Candidato:

Gianluca Mannini

Mat. 05121 05680

ANNO ACCADEMICO 2019/2020

INDICE

1	INTRODUZIONE	4
2	RELATED WORKS	7
3	BACKGROUND	12
3.1	Natural User Interfaces	12
3.2	Il Motore Grafico Unity 3D	14
3.2.1	Assets	17
3.2.2	GameObjects & Components	17
3.2.3	Scene	18
3.2.4	Scripting: La classe MonoBehaviour	18
3.2.5	Audio in Unity	21
3.3	Fondamenti di Acustica	22
3.4	Digital Audio	23
3.5	Gesture Recognition	26
3.6	Leap Motion	27
3.7	Artificial Neural Networks	28
3.8	Paradigmi di apprendimento	30
4	MARCOSMILES 2.0	32

5	MODULO SONORO: L'APPROCCIO UTILIZZATO	34
5.1	Ipotesi Implementative & Analisi Costi-Benefici	34
5.1.1	Triggers Di Clip Audio	35
5.1.2	Interfacciamento con la tecnologia MIDI	36
5.1.3	Generazione del suono in modo procedurale utilizzando codifiche PCM	41
5.1.4	Conclusioni ipotesi illustrate	44
5.2	Generazione forme d'onda elementari: Un primo approccio . . .	45
5.2.1	Sine Wave	45
5.2.2	Square Wave	49
5.2.3	Sawtooth Wave	51
5.3	Un primo esempio di Sintetizzatore	53
5.4	Sintesi Additiva: Un nuovo approccio	59
5.4.1	La classe SinusWave	60
5.4.2	La classe SquareWave	64
5.4.3	La classe SawWave	68
5.5	Sintetizzatore in MarcoSmiles	71
5.5.1	Implementazione Finale & Interfacciamento con gli altri Moduli	76
6	MODULAZIONI & EFFETTI	83
6.1	Frequency Modulation	83
6.2	Amplitude Modulation	85
6.3	Riverbero, Chorus & Filtro Passa Basso	86
7	GUI & OSCILLOSCOPIO	87
7.1	GUI Sintetizzatore	88
7.2	Oscilloscopio	89
8	CONCLUSIONI	92
8.1	Risultati e Discussioni	92
8.2	Lavori futuri	94

CAPITOLO 1

INTRODUZIONE

Viviamo in un'epoca in cui la tecnologia ha completamente rivoluzionato molti degli aspetti della vita giornaliera di un individuo. In un contesto simile fornire accesso agli strumenti informatici di uso comune ad ogni tipo di utente è di fondamentale importanza. Requisiti come l'usabilità, intesa come la facilità di interazione tra l'uomo e uno strumento, e accessibilità, ovvero la caratteristica di un servizio di essere fruibile da tutte le categorie di utenti, rappresentano dei punti cardine in fase di progettazione di un sistema informatico, e sono oggetto di studio della HCI (Human Computer Interaction). Il costante sviluppo in questi ambiti ha portato alla nascita di strumenti che possano permettere l'utilizzo dei dispositivi elettronici a chiunque, scavalcando barriere derivanti da problemi quali il gap inter-generazionale, difficoltà d'utilizzo, mancanza di accessibilità e scarsa attrattività. In particolare negli ultimi anni sono stati esplorati nuovi modi di interagire con i computer, progettando nuove interfacce basate sull'utilizzo di ambienti 3D, gestures e contextual awareness, al fine di rendere l'interazione più semplice, intuitiva e accattivante. Un altro ambito, oggetto di studi negli scorsi anni, è stato quello delle Natural User Interfaces (NUI), descritte alla presentazione della conferenza "Predicting the Past" del 2008 come "la

1. INTRODUZIONE

prossima fase evolutiva dopo il passaggio dall'interfaccia a riga di comando (CLI) all'interfaccia grafica utente (GUI)” [2]. L'obiettivo principale delle NUI è quello di superare le classiche interazioni fisiche legate all'utilizzo di mouse e tastiera, per sostituirle con un più ampio insieme di modalità di comunicazione innovative, che prevedano l'utilizzo di azioni, gesti e movimenti naturali da parte dell'utente, per controllare la macchina che sta utilizzando. La popolarità di queste interfacce innovative è in continua crescita, soprattutto grazie alla diffusione di dispositivi low-cost per il tracking dei movimenti, come il Leap Motion o il Kinect, e dispositivi che supportano il riconoscimento della voce come Amazon Alexa. Nonostante la già grande popolarità nel mercato dei videogiochi, queste tecnologie sono state impiegate in vari settori quali la riabilitazione, la robotica, e l'home automation. L'obiettivo del nostro lavoro è quello di utilizzare questo tipo di interfacce, insieme ad un dispositivo per il riconoscimento dei movimenti delle mani, nell'ambito della *Computer Music*. La computer music o musica informatica è un ramo della musica elettroacustica che impiega sistematicamente la programmazione su elaboratore numerico. In senso più esteso, qualunque produzione musicale che utilizzi il mezzo informatico può definirsi computer music [3]. Pietro Grossi, pioniere italiano dell'ingegneria, nel 1967, alle prime esperienze ufficiali di computer music presso la Olivetti General Electric a Pregnana Milanese dichiarò: “. . . Ho preso il quinto Capriccio di Paganini, ho perforato le schede e, fatto il pacco di schede, consegnate al computer, il computer le suonava. Il computer ha suonato subito alla perfezione il testo che gli avevo dato, e immediatamente dopo, ho potuto fare di quei suoni quello che volevo. Questo per me era un salto incredibile. È stata un'emozione straordinaria. . . ” [4]. Quest'ambito è in continua evoluzione e, negli ultimi anni, diversi musicisti hanno mostrato interesse nella possibilità di utilizzare dei controller che siano in grado di personalizzare il modo in cui ci si esprime “musicalmente”. Un esempio degno di nota è la performance eseguita dalla nota cantante americana Ariana Grande nel 2015 [5], in cui fa uso dei guanti Mi.Mu Gloves [6] per armonizzare la melodia che sta cantando, mettendo in luce le grandi potenzialità derivanti dall'utilizzo di un controller

1. INTRODUZIONE

per l'espressività. Per la maggior parte degli strumenti musicali reali, è possibile rappresentare le note musicali in termini di configurazioni della mani. Ogni configurazione contiene informazioni circa la posizione e la direzione di mano e dita, per ciascuna frazione temporale. Una modalità semplice e naturale di associare gesti dell'utente alla riproduzione di musica real-time è proprio attraverso l'associazione di configurazioni delle mani a delle specifiche note musicali. Su questo principio è basato il nostro sistema MarcoSmiles, uno strumento rivolto principalmente a persone affette da disabilità motorie, ma utilizzabile anche da chi è interessato all'ambito musicale o semplicemente per attività di intrattenimento, che consente di poter suonare uno strumento virtuale attraverso dei semplici gesti delle mani, senza l'uso di uno strumento musicale reale e in maniera del tutto personalizzabile. Per il funzionamento del sistema è necessario un sensore LeapMotion che avrà il ruolo di acquisire informazioni circa la posizione della mani dell'utente, per poi fornirle ad una Rete Neurale Artificiale (ANN) opportunamente addestrata che provvederà a realizzare l'associazione posizione - nota musicale real-time. Questo lavoro di tesi ha come obiettivo quello di superare i limiti di tale progetto, in particolare per quello che concerne la parte della generazione del suono *real-time*.

Nel Capitolo 2 discuteremo di alcuni lavori rilevanti nel settore musicale, tracciando un percorso storico sull'evoluzione degli strumenti musicali non convenzionali. Nel Capitolo 3 verranno fornite nozioni di *background* e descritti i principali argomenti che verranno trattati in questo lavoro, con lo scopo di illustrare successivamente il lavoro di tesi in maniera cogente. Nel quarto capitolo verrà fornita una descrizione del sistema e degli obiettivi che si pone. Nel quinto capitolo verranno analizzate le ipotesi implementative e ripercorsi gli *step* effettuati durante la progettazione e lo sviluppo del motore sonoro. Nel Capitolo 6 verranno analizzate le tecniche utilizzate per l'implementazione di modulazione ed effetti sul suono. All'interno del settimo capitolo verrà illustrata e spiegata l'interfaccia grafica del sintetizzatore costruito. Infine, nell'ottavo capitolo verranno discussi i risultati ottenuti, e le possibili prospettive sui lavori futuri per estendere il sistema.

CAPITOLO 2

RELATED WORKS

Negli ultimi anni sono stati effettuati molti studi che hanno evidenziato una vasta scelta di possibilità nel produrre musica in maniera non convenzionale, attraverso strumenti elettronici sia fisici che virtuali. Tuttavia per comprendere al meglio la storia e l'evoluzione degli strumenti musicali accessibili è necessario fare un passo indietro; quando l'utilizzo dell'elettricità era ancora nelle prime fasi di sperimentazione, e la lampadina doveva ancora essere inventata, alcune persone erano già a lavoro per utilizzare l'elettricità nel processo di creazione del suono. A causa della mancanza delle tecnologie di registrazione, questi primi esperimenti furono progettati esclusivamente per performance live [8]. Nel 1919 è nato il *Theremin* [9], il primo strumento che non richiede contatto fisico con l'esecutore. Il Theremin è stato inventato dal fisico sovietico Lev Sergeevič Termen nel 1919, e si basa sull'utilizzo di oscillatori elettronici che lavorano in isofrequenza al di fuori dello spettro udibile. Il musicista provoca un'alterazione degli oscillatori inserendo le sue mani nel campo d'onda, producendo dei suoni sul principio fisico del battimento, questa volta nel campo delle frequenze udibili. Lo strumento è sostanzialmente composto da 2 antenne annesse ad un contenitore in cui alloggia l'elettronica dello strumento. Le antenne controllano rispettivamente

2. RELATED WORKS

frequenza e intensità del suono prodotto. Il Theremin ha rappresentato una vera e propria rivoluzione nell'ambito della musica, ed è stato utilizzato negli anni successivi in diversi contesti e in numerosi generi musicali.

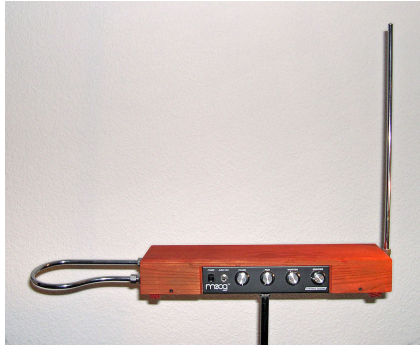


Figura 2.1: Theremin

Verso la prima metà degli anni 70 è stato sviluppato l'iconico *Mellotron* [10], uno strumento a tastiera pensato per riprodurre strumenti musicali tipici delle orchestre sinfoniche, e voci umane. Questo strumento può essere considerato come il padre dei campionatori elettronici, comparsi all'inizio degli anni 80. Il mellotron genera i suoni tramite nastri registrati. Quando viene premuto un tasto, il nastro collegato viene spinto sulla testina di riproduzione, come in un registratore a nastro. Finché il tasto resta abbassato, il nastro scorre sulla testina e il suono viene riprodotto. Quando il tasto viene rilasciato, una molla fa tornare indietro il nastro alla sua posizione di riposo. In seguito ai progressi dell'industria elettronica, furono prodotti numerosi strumenti elettronici di vario genere, tra i quali possiamo ricordare il *Suzuki Omnichord* [11]. Il suo funzionamento è semplice: la mano sinistra preme i tasti che formano gli accordi, mentre la destra sfiora il touchpad che sostituisce le corde. L'omnichord offre altre possibilità per ottenere suoni, anche utilizzando soltanto la tastiera. È uno dei primi strumenti elettronici ad essere considerato accessibile, e viene utilizzato tutt'ora in musicoterapia. Attualmente nell'ambito della Computer Music rivestono un ruolo di grande importanza sensori di varie tipologie. Tra i più utilizzati troviamo i sensori tattili, fotocamere e accelerometri. Un importante esempio è rappresentato

2. RELATED WORKS

dal guanto *KAiKU* [13], un sensore capacitivo indossabile, che risponde alla prossimità e al contatto di conduttori, come il corpo umano.



Figura 2.2: Kaiku Glove durante una performance musicale

Il sensore viene principalmente utilizzato come controller MIDI, quindi può essere utilizzato in vari contesti, e dispone di un software dedicato che permette di emulare diversi tipi di strumenti. Un tipico esempio di utilizzo consiste nel mappare delle note a delle zone del guanto, che possono essere sfiorate con le dita o altri oggetti conduttori per inviare un segnale MIDI al dispositivo a cui è connesso il guanto, in modo che venga suonata la nota desiderata. Sono presenti anche altri tipi di sensori indossabili, e vengono spesso utilizzati per la riabilitazione e la musicoterapia. Un altro strumento usato in questi due ambiti è il software *AUMI* [12]. Quest'ultimo permette all'utente di poter riprodurre suoni e frasi musicali attraverso movimenti e gestures, sfruttando la fotocamera del dispositivo sul quale è in uso. Il software prevede un approccio personalizzabile attraverso l'utilizzo di effetti e la possibilità di variare i suoni utilizzati. Il sistema è progettato principalmente per essere utilizzato da bambini che hanno disabilità fisiche. Molto diffuso è anche il *Kinect* [14], un sensore sviluppato da Microsoft per le console Xbox. Il Kinect si avvale di una telecamera RGB e di una telecamera a radiazione infrarossa con una risoluzione di 480p ciascuna, ed una capacità di lettura del movimento a 30fps (frames per second). Per il calcolo della lontananza degli oggetti, invece, Kinect utilizza di uno scanner 3D, in combinazione con gli infrarossi, che però è dotato di una risoluzione di 320×240 pixel. Kinect dispone inoltre di quattro microfoni utilizzati dal sistema per l'elaborazione dati dell'ambiente in cui ci si trova, che permettono

2. RELATED WORKS

la cancellazione dell'eco e riduzione del rumore. Il kinect è stato utilizzato in numerosi progetti di vario genere nell'ambito della Computer Music, spesso facendo uso dell'Hand Gesture Recognition. Un esempio consiste nel lavoro effettuato dalla National Taiwan University of Science and Technology [15], ovvero un sistema capace di simulare la conduzione musicale in maniera real-time, sfruttando un classificatore. Il sensore utilizzato nel nostro sistema è il *LeapMotion Controller*. L'applicazione del sensore LeapMotion nell'ambito della Computer-Music è stata oggetto di diversi studi. In primo luogo è stata esplorata la possibilità di utilizzare il LeapMotion assieme al protocollo *MIDI*, sfruttando la Gesture-Recognition al fine di inviare segnare segnali per modulare determinati parametri all'interno di una DAW o di VST. Un esempio di questo tipo di applicazione è lo strumento AeroMIDI [16], che permette di creare un' interfaccia completamente personalizzabile, in cui è possibile mappare parametri di un Virtual Instrument o di qualsiasi altra funzione, a gesti delle mani, con la possibilità di controllarne fino a 6 per mano. Il sistema AeroMIDI fa parte della categoria dei numerosi strumenti che utilizzano il LeapMotion come controller MIDI, e garantiscono all'utente una maggiore espressività durante la performance musicale. Uno studio sull'Gesture Recognition utilizzando il LeapMotion è stato effettuato dalla Dalian University of Technology [17], e ha evidenziato la possibilità di sviluppare un classificatore in grado di riconoscere accuratamente gesture scelte dall'utente in fase di training. Un altro progetto che utilizza il LeapMotion nell'ambito della Computer - Music è Virtual Piano [18]. Si tratta di un software basato sulla gesture recognition, che permette di poter suonare un pianoforte virtuale attraverso delle gesture delle mani predefinite, e dispone di alcune semplici canzoni da poter suonare.

2. RELATED WORKS

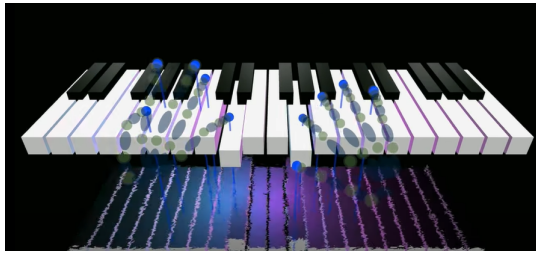


Figura 2.3: Virtual Piano

E' importante sottolineare che la principale differenza fra i sistemi sopracitati rispetto a MarcoSmiles è che la configurazione delle mani associata ad una determinata nota è scelta dall'utente in base alle sue preferenze. L'utente dovrà allenare il sistema ad imparare i gesti che intende utilizzare, per poi successivamente utilizzare per suonare in modo naturale, come se fosse uno strumento reale.

CAPITOLO 3

BACKGROUND

3.1 Natural User Interfaces

Come già accennato nel paragrafo introduttivo, le Natural User Interfaces sono interfacce invisibili all'utente, caratterizzate da interazioni semplici e di facile apprendimento. La parola “Natural” si riferisce proprio all'obiettivo delle NUI, cioè rendere l'interazione con l'utente del tutto naturale, attraverso l'impiego di facoltà umane quali tatto, vista, utilizzo della voce, movimento, fino ad arrivare a funzioni cognitive più complesse come percezione ed espressione. Un altro aspetto importante relativo le NUI consiste nella selezione di gestures che l'utente conosce e utilizza durante la vita di tutti i giorni. Ciò avviene in modo da ridurre il carico cognitivo dell'utilizzatore, cioè lo sforzo richiesto nel pensare o effettuare un ragionamento prima di usare una determinata gesture all'interno dell'interfaccia, in maniera tale da eliminare qualsiasi tipo di distrazione. Da tenere in grande considerazione è anche il contesto in cui la NUI viene utilizzata. Durante la fase di progettazione bisogna tener conto dell'impossibilità nel creare un' interfaccia che risulti naturale in tutti i contesti e per tutti gli utenti. Di conseguenza la scelta delle gestures deve essere effettuata alla luce del contesto in cui l'interfaccia viene utilizzata,

3. BACKGROUND

e tenendo conto dello skill level dell'utilizzatore medio. Il noto fondatore di Microsoft, Bill Gates, ha dichiarato in merito alle NUI: “Fino ad ora ci siamo sempre dovuti adattare ai limiti imposti della tecnologia e adattare il modo in cui lavoriamo con i computer ad un insieme di procedure e convenzioni. Attraverso le NUI, per la prima volta saranno i computer ad adattarsi ai nostri bisogni e preferenze, e per la prima volta gli esseri umani interagiranno con le macchine in maniera del tutto naturale” [2].

3.2 Il Motore Grafico Unity 3D

Il nostro sistema è basato su Unity3D[25], un motore grafico multi-piattaforma che consente lo sviluppo di videogiochi e altri contenuti interattivi, quali visualizzazioni architettoniche o animazioni 3D in tempo reale. Unity permette all'utente di creare ambienti sia in 2D che in 3D. Uno screening del 2018 rileva che Unity è stato utilizzato per la creazione di circa la metà dei giochi mobile disponibili sul mercato e del 60% dei giochi in realtà aumentata e VR. È inoltre stato adottato da altri ambiti industriali che non rientrano in quello del settore dei videogiochi, come cinematografica, architettura, ingegneria e costruzione. Un vantaggio non di poco conto è che Unity mette a disposizione versioni gratuite per studenti e per privati con entrate inferiori ai 100.000\$ annuali.

L'Engine Unity3D, rende agevole la disposizione di livelli, la creazione di menu, l'esecuzione di animazioni e l'organizzazione dei progetti. Inoltre, integra al proprio interno la possibilità di scrivere ed eseguire scripts, scritti in C#.

L'interfaccia grafica è molto ben organizzata e i pannelli di lavoro sono totalmente personalizzabili. E' proprio attraverso la selezione ed il trascinamento dei pannelli che si attiva tale personalizzazione.

3. BACKGROUND

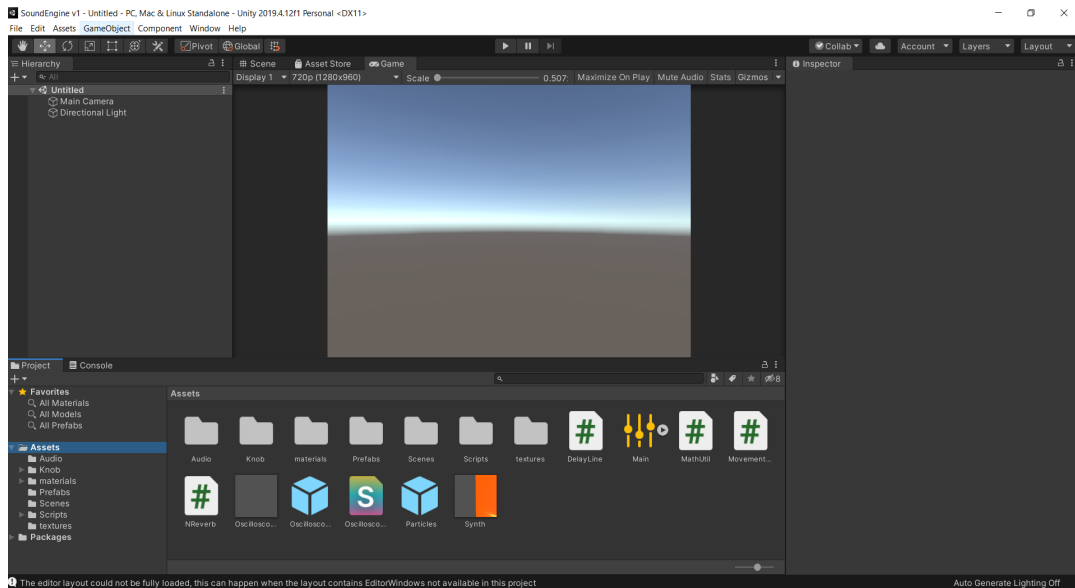


Figura 3.1: Scena vuota Unity3D

Questi i principali pannelli all'interno di Unity:

- Il pannello “Project” è quello in cui tutti gli assets del progetto vengono immagazzinati. Da qui è possibile organizzare i file che usiamo all'interno di un progetto.

Ogni volta che importiamo degli assets appariranno qui.

- Il pannello “Hierarchy” è quello in cui gli assets vengono organizzati all'interno di una scena. Ogni Asset, può essere trasportato dal pannello “Project” a quello “Hierarchy”, in modo tale da inserire assets all'interno di una scena. Gli elementi all'interno di questo pannello sono chiamati GameObjects.

- Il pannello “Inspector” permette di ispezionare e regolare tutti gli attributi di un GameObject. Da questo pannello si possono regolare per un GameObject, ad esempio, la posizione, la rotazione, le proprietà di gravità all'interno del gioco, e tutte le altre caratteristiche.

Questo è fatto attraverso l'ausilio di Components, ossia, componenti che ci permettono di specificare determinate proprietà per il GameObject selezionato. Anche gli scripts vengono trattati come se fossero Components e, dunque, aggiunti tramite questo pannello.

3. BACKGROUND

- Il pannello “Scene” permette di osservare e di arrangiare gli assets all’interno del gioco e dello spazio 3D. Da qui, è possibile spostarci all’interno dello spazio 3D utilizzando il mouse.
- Il pannello “Console” è letteralmente la console di Unity. Molto utile in fase di debugging degli scripts.

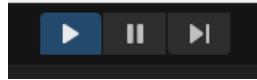


Figura 3.2: Pulsanti Play

Per far partire il proprio gioco si preme sul pulsante Play posto nella parte alta della schermata. Si può mettere in pausa il gioco (usando il pulsante Pause) in modo tale da ispezionare la scena. Inoltre è possibile passare allo step successivo del gioco, utilizzando il pulsante Step.

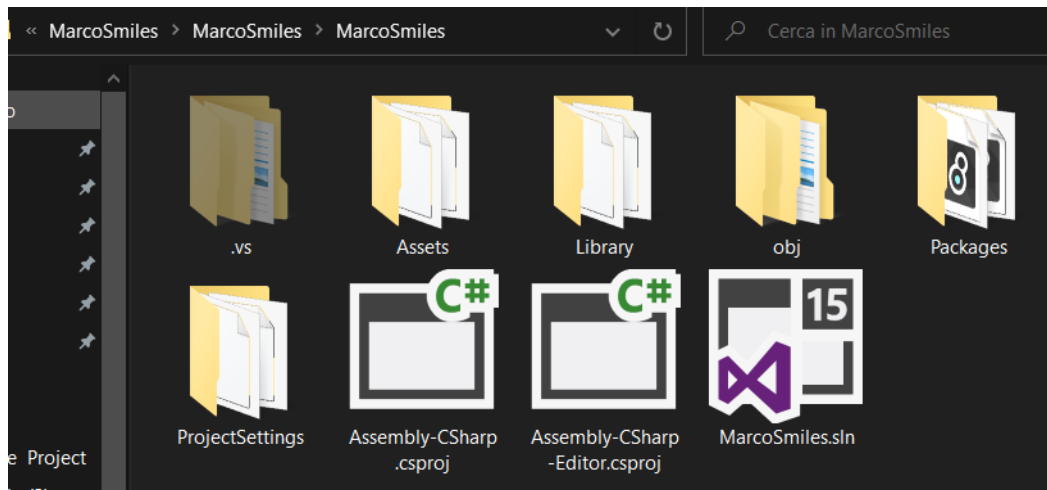


Figura 3.3: La directory di un progetto Unity

In Unity, un progetto è una cartella ordinaria, contenente ogni risorsa utilizzata dal gioco. Infatti, come risultato della creazione di un progetto nuovo, avremo una cartella che contiene delle sottocartelle chiamate rispettivamente “Assets”, “Library” e “ProjectSettings”.

3.2.1 Assets

Gli Assets sono una qualsiasi risorsa utilizzata dal gioco. Questo include modelli 3D, Materials, Textures, Audio, Scripts, Fonts etc..

Unity gestisce in modo robusto ed intelligente l'importazione di Assets in un progetto. Gestisce al meglio la maggior parte dei formati di file, risolvendo uno dei difetti che tradizionalmente ha caratterizzato vecchi game engine, ossia quello di essere vincolati all'estensione del file quando si cerca di importarlo all'interno di un progetto.

Unity ha funzionalità built-in che permettono di creare semplici oggetti come Sfere o Cubi, ma quando c'è la necessità di lavorare con oggetti più complessi, l'ideale è importare modelli 3D all'interno del proprio progetto. L'Engine riesce a gestire la maggior parte dei formati 3D, inclusi Maya, 3D Studio Max, Blender e FilmBox, mantenendo tutti i materiali e le textures intatte. Stesso discorso è valido per ciò che riguarda formati di immagini, supportando PNG, JPEG, TIFF e addirittura file PSD con livelli Photoshop, e per i formati di file audio, supportando formati quali WAV, AIFF, MP3 e OGG.

Una lista completa di tutti i formati che possono essere importati all'interno di Unity la si può trovare al seguente link:

<http://unity3d.com/unity/editor/importing>

E' opportuno notare che Unity mette a disposizione un Assets Store, dove si possono acquistare o scaricare gratuitamente Assets e risorse per i propri progetti.

3.2.2 GameObjects & Components

Il concetto di **GameObject** è probabilmente il concetto più importante all'interno dell'editor di Unity. Ogni oggetto all'interno del gioco è un **GameObject**. Nonostante ciò, un **GameObject** non può fare nulla all'interno del gioco se non gli si conferiscono proprietà. Possiamo descrivere un **GameObject** come una scatola che permette di contenere funzionalità. Queste funzionalità sono descritte dai **Components**. A seconda di che tipo di

3. BACKGROUND

oggetto si vuole creare, si aggiungono diverse combinazioni di **Components** all'interno di un **GameObject**.

Un **GameObject** contiene sempre una Component di tipo Transform che non è possibile rimuovere. Questa rappresenta la posizione e l'orientamento dell'oggetto.

Altre Components che danno funzionalità agli oggetti possono essere aggiunte dal menu rinominato “Component” o da uno script.

E' importante notare che esistono degli oggetti pre-costruiti chiamati Primitive Objects. Questi si trovano nel menu GameObject/3D Object e permettono di creare degli oggetti semplici come una camera, una sfera o un cilindro.

Per default, ogni scena viene inizializzata con un GameObject chiamato Main Camera. Questo oggetto è configurato per comportarsi come camera principale all'interno del gioco. Contiene una componente di tipo Transform, una di tipo Camera e un'altra di tipo AudioListener che permette di percepire i suoni che si trovano nell'ambiente 3D di una scena.

3.2.3 Scene

Le scene rappresentano determinati ambienti del gioco. Sono i “luoghi” che contengono gli Assets e descrivono la schermata del gioco. Mentre il pannello “Scene” è ideale per arrangiare gli Assets all'interno dello spazio 3D, il pannello “Hierarchy” è adatto ad organizzare i GameObjects all'interno della scena, descrivendo i contenuti della scena corrente tramite relazioni padre-figlio fra i vari GameObjects.

3.2.4 Scripting: La classe MonoBehaviour

Gli scripts sono anche chiamati “behaviours” in Unity. Permettono di rendere gli assets ed i GameObjects interattivi all'interno di una scena. Più script possono essere collegati ad uno stesso GameObject, permettendo un facile riuso di codice.

Unity supporta lo scripting in C#.

3. BACKGROUND

La classe **MonoBehaviour** è la classe base da cui deriva ogni script in Unity.

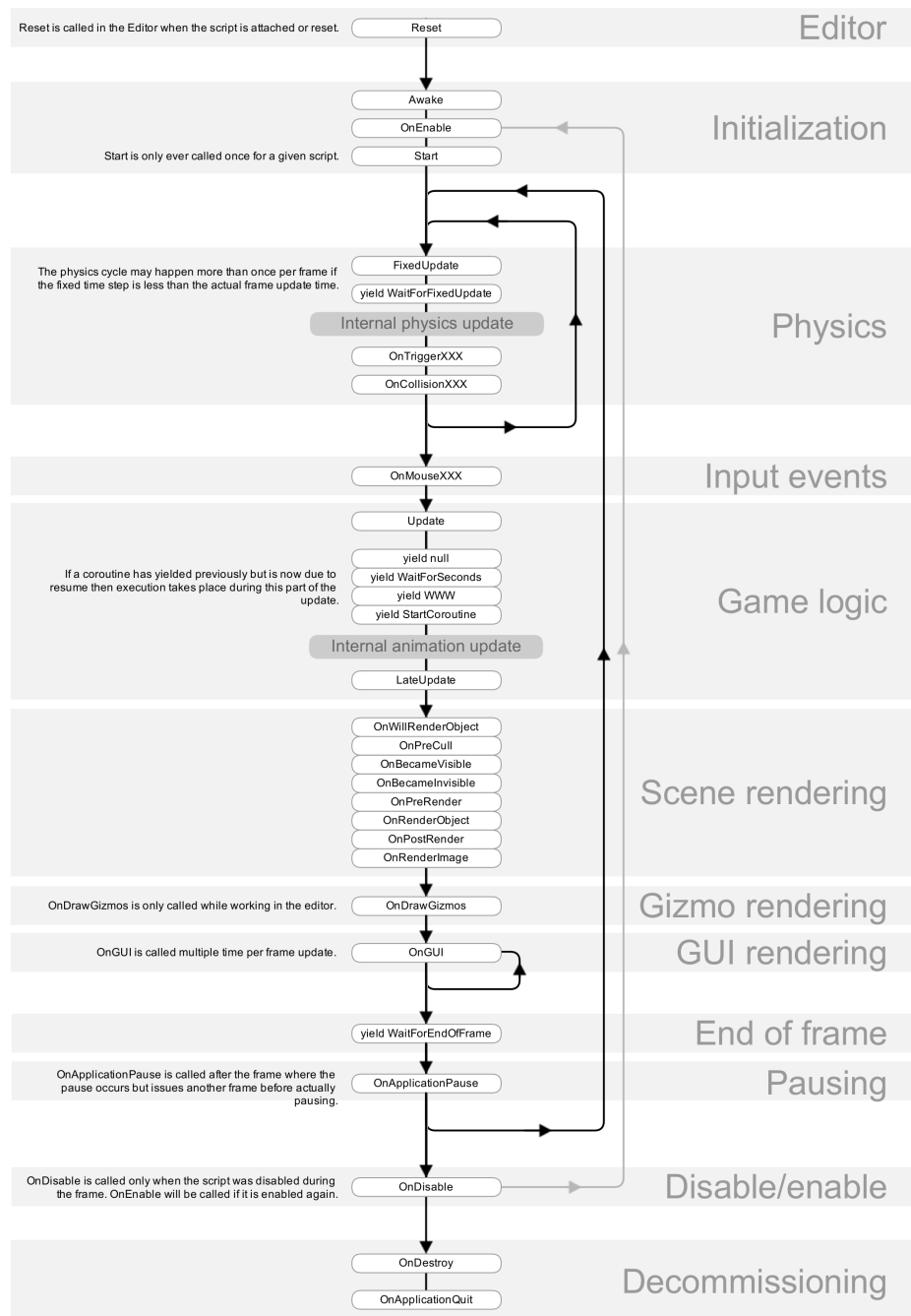


Figura 3.4: Diagramma metodi classe MonoBehaviour [26]

3. BACKGROUND

Quando si crea uno script dal pannello “Project”, lo script eredita in automatico dalla classe **MonoBehaviour**. La classe MonoBehaviour mette a disposizione un framework che permette di collegare gli script a GameObjects all’interno dell’editor, inoltre permette di intercettare determinati eventi relativi al ciclo di vita degli oggetti, in modo da facilitare lo sviluppo delle applicazioni e da eseguire il proprio codice sulla base di quello che sta avvenendo all’interno del progetto.

Di seguito si elencano alcune delle funzioni più importanti della classe **MonoBehaviour**:

- **Awake:** Chiamato quando l’istanza dello script è caricata. E’ principalmente usato per inizializzare variabili o stati, prima che l’applicazione inizi.
- **Start:** Simile al metodo Awake. Viene chiamato ogni volta che un GameObject inizia ad esistere (quando la scena è caricata, oppure quando il GameObject è inizializzato). A differenza del metodo Awake, il metodo Start non può essere chiamato allo stesso frame di Awake se lo script non è stato attivato a tempo di inizializzazione. Questa temporizzazione tra il metodo Awake e il metodo Start risulta essere molto utile in casi in cui esistono due oggetti A e B e l’oggetto A ha bisogno di utilizzare informazioni contenute nell’oggetto B. In questo caso, l’inizializzazione di B dovrebbe essere fatta in Awake, mentre quella di A in Start.
- **Update:** Chiamato ad ogni frame, se lo script MonoBehaviour risulta attivo. Questo è il metodo più usato per implementare funzionalità di scripting di qualsiasi tipo.
- **FixedUpdate:** Simile al metodo Update, ma è indipendente dal Frame-Rate utilizzato dall’ applicazione. Generalmente, viene chiamato più spesso del metodo Update (ma non in ogni caso!).

- **OnDestroy**: chiamato ogni volta che un `GameObject` viene “distrutto”.
- **StartCoroutine**: Inizia una coroutine. L’esecuzione della coroutine può essere fermata in qualsiasi punto del codice usando lo statement *yield*
- **OnAudioFilterRead**: Unity chiama automaticamente questo metodo ogni volta che un oggetto `AudioSource` cerca di riprodurre un suono. ***OnAudioFilterRead*** riceve un array di floats chiamato ***data*** come parametro, questo array sono i campionamenti PCM che l’oggetto `AudioSource` sta riproducendo. Il secondo parametro ***channels*** rappresenta il numero di canali disponibili che stanno riproducendo il suono.

3.2.5 Audio in Unity

Nella vita reale, i suoni sono generati da oggetti e ascoltati attraverso le orecchie delle persone. Il modo in cui il suono è percepito dipende da molti fattori. Una persona che sta ascoltando un suono riesce a percepire approssimativamente da che direzione e da che lontananza il suono proviene. Un oggetto che si muove velocemente emettendo un suono cambierà di tono mentre si muove, come risultato dell’effetto Doppler. Così come l’ambiente all’interno del quale il suono è stato prodotto cambierà il suono stesso, producendo ad esempio eco.

Per simulare l’effetto della posizione del suono, Unity ha bisogno che i suoni siano generati da una componente **AudioSource**, collegata ad un oggetto in una Scena. I suoni emessi vengono poi percepiti da una componente **AudioListener** collegata ad un altro oggetto (di solito alla camera principale della Scena). In questo modo Unity può simulare l’effetto della distanza e della posizione tra l’oggetto `Audio Source` e l’oggetto `Audio Listener` e riprodurre il suono di conseguenza.

Unity non riesce a calcolare gli echi e gli effetti puramente partendo dalla Scena, ma questi si possono simulare utilizzando **AudioFilters**.

AudioSource: Un oggetto **Audio Source** permette di riprodurre una clip audio all'interno di una Scena. Questa clip, può essere riprodotta verso un Audio Listener, oppure attraverso un Audio Mixer. Un **Audio Source** può riprodurre qualsiasi tipo di clip audio e può essere configurato per riprodurre le clip in modalita 2D, 3D, o una via di mezzo (SpatialBlend).

AudioListener: Un oggetto **Audio Listener** si comporta come se fosse un microfono all'interno di una Scena. Riceve input da ogni Audio Source nella scena e riproduce i suoni attraverso le casse per computer. Solitamente un Audio Listener viene collegato alla camera principale della scena.

AudioFilters: Un oggetto **Audio Filter** permette di modificare l'output di **Audio Source** o di **Audio Listener**, applicando degli effetti audio. Questi possono permettere di filtrare in suono, oppure di applicare riverbero o altri effetti. Gli effetti sono applicati aggiungendo componenti nell'oggetto contenente l'**Audio Source** o l'**Audio Listener** che sta generando il suono. L'ordine in cui queste componenti vengono collegate agli oggetti è importante, in quanto definirà l'ordine in cui gli effetti andranno a modificare l'audio.

3.3 Fondamenti di Acustica

Il suono viene creato da turbolenze ondulatorie emesse da una “sorgente sonora” e propagate nell'ambiente circostante (solitamente l'aria). Si tratta di onde di pressione che vengono percepite come suoni dall'orecchio umano. Le onde sonore possono essere rappresentate graficamente utilizzando un grafico cartesiano, riportante il tempo, (t) sull'asse delle ascisse, e gli spostamenti delle particelle, (s) su quello delle ordinate. Il tracciato esemplifica gli spostamenti delle particelle: alla fine, la particella si sposta dal suo punto di riposo (asse delle ascisse) fino al culmine del movimento oscillatorio, rappresentato dal ramo crescente di parabola che giunge al punto di massimo parabolico. Poi la particella inizia un nuovo spostamento in direzione opposta, passando per il punto di riposo e continuando per inerzia

3. BACKGROUND

fino ad un nuovo culmine simmetrico al precedente, questo movimento è rappresentato dal ramo decrescente che, intersecando l'asse delle ascisse, prosegue in fase negativa fino al minimo parabolico. Infine, la particella ritorna indietro e ripete nuovamente la sequenza di spostamenti, così come fa il tracciato del grafico.

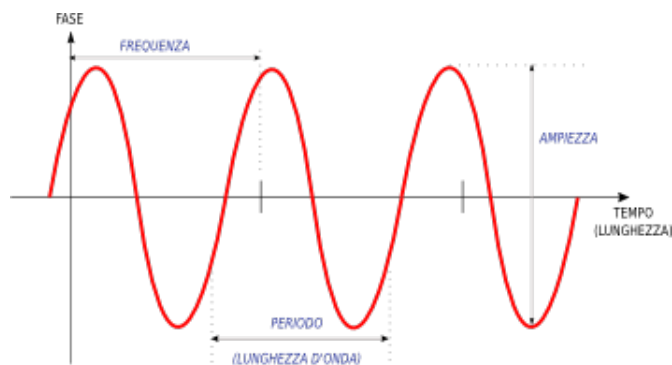


Figura 3.5: Rappresentazione di una Sinusoide

Il periodo (graficamente il segmento tra due creste) è il tempo impiegato dalla particella per tornare nello stesso punto dopo aver cominciato lo spostamento (indica cioè la durata di una oscillazione completa). La distanza dalla cresta all'asse delle ascisse indica, invece, l'ampiezza del movimento. In acustica, si preferisce solitamente usare altre grandezze derivate da queste ultime. Il numero di periodi compiuti in un secondo esprime la frequenza in hertz (Hz). Dall'ampiezza dell'onda, invece, si calcola la pressione sonora, definita come la variazione di pressione rispetto alla condizione di quiete, e la potenza e l'intensità acustica, definita come il rapporto tra la potenza dell'onda e la superficie da essa attraversata; l'intensità delle onde sonore viene comunemente misurata in decibel.

3.4 Digital Audio

Il suono viene creato da turbolenze ondulatorie emesse da una “sorgente sonora” e propagate nell'ambiente circostante (solitamente l'aria). Si tratta

3. BACKGROUND

di onde di pressione che vengono percepite come suoni dall'orecchio umano. Per convertire un'onda sonora analogica in un segnale digitale, il computer deve essere in grado di misurarne l'ampiezza in determinati punti. Ogni misura rilevata viene chiamata *campione*, perciò la conversione da analogico a formato digitale prende il nome "campionamento del suono".

Il teorema del campionamento di **Nyquist-Shannon** afferma che, se la frequenza di campionamento è sufficientemente grande, non si hanno perdite di informazione rispetto alla forma d'onda originale. Dunque, più ampia sarà la frequenza di campionamento, tanto maggiore sarà anche il numero delle misurazioni d'ampiezza e, quindi, la precisione del segnale digitale.

Se viene usata una **frequenza di campionamento** (*sample rate*) alta, la serie di numeri prodotta contiene pressoché intatta tutta la informazione sulla forma d'onda elettrica analogica iniziale. Nei moderni standard tecnologici, in genere le frequenze di campionamento spaziano dagli 8.000 campioni al secondo (Samples per second, S/s) per la voce telefonica, fino ai 44.100 e più campioni al secondo per la qualità musicale. Queste letture di valori di tensione possono poi cadere in un qualsiasi punto della dinamica del segnale, cioè ogni singolo campione può avere un valore compreso tra il minimo e il massimo possibile.

Siccome potenzialmente si possono avere infiniti valori di lettura di tensione per ogni singolo campione, per completare l'opera di conversione del segnale da analogico in digitale, va suddiviso tutto il possibile range dinamico del segnale in un numero finito di intervalli e ogni singolo intervallo va codificato con un valore digitale ben determinato.

Queste due operazioni si chiamano quantizzazione e codifica di sorgente. La quantizzazione in genere suddivide il range dinamico del segnale in un numero di intervalli potenza del due (2^n *intervalli*), in maniera tale che ogni singolo campione cadrà inevitabilmente in uno degli intervalli quantizzati e potrà così essere codificato digitalmente con n bit. I valori più ricorrenti di digitalizzazione attualmente usati vanno da un minimo di 8 bit per campione in campo telefonico (range dinamico del segnale suddiviso in 256 intervalli),

3. BACKGROUND

fino a 20 e più bit per campione (range dinamico del segnale suddiviso in un milione e più di intervalli).

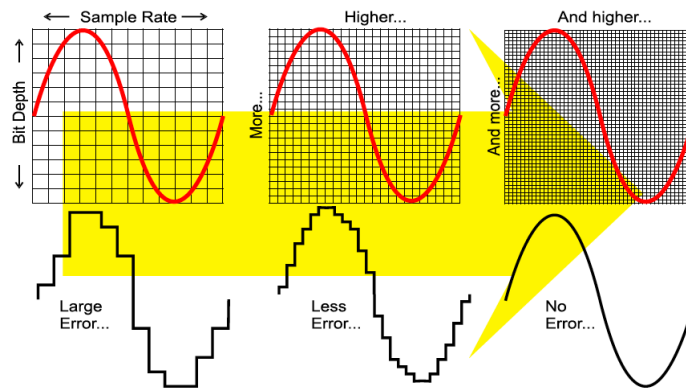


Figura 3.6: Margine di errore per frequenze di campionamento crescenti [21]

Frequenza di Campionamento. La **frequenza di campionamento** è il numero di misurazioni fatte al secondo per convertire un segnale analogico in digitale. Aumentando le misurazioni al secondo, siamo in grado di raccogliere più informazioni reali, e dunque di ricostruire un'immagine del suono molto accurata.

Profondità di Bit. La **profondità di bit** permette di definire il range dinamico del segnale. A ogni campionamento è assegnato un'ampiezza specifica. Una **profondità di bit** elevata, permetterà di avere più spazio tra volumi alti e volumi bassi, migliorando così il range dinamico dei campionamenti e della registrazione. In generale, per ogni bit che si aggiunge alla **profondità di bit** utilizzata, il range dinamico aumenta di 6dB.

Codifica Digitale PCM. La **codifica digitale PCM** (abbreviazione che sta per Pulse Code Modulation) è il nome generico per il sistema di registrazione digitale. E' la forma standard in cui viene trattato l'audio digitale in computer, CD, telefonia digitale e altre applicazioni di audio digitale. Nel testo, useremo il termine “**codifiche PCM**” per riferirci ad una sequenza di campionamenti digitali che descrivono un segnale analogico.

3.5 Gesture Recognition

I metodi di interazione in input tra uomo e macchina, di base, sono mouse e tastiera, che non sempre risultano confacenti agli utenti meno avvezzi all'uso di un computer, o addirittura impossibili per gli utenti portatori di *handicap motori*. È per questo che nel corso degli anni sono stati studiati nuovi metodi alternativi per interagire con un computer.

Agli inizi degli '80 nasceva quindi un primo metodo di input alternativo, lo *Speech Recognition*, che permetteva ad un utente di inserire del testo semplicemente parlando. Col passare degli anni lo *Speech Recognition* è stato ulteriormente sviluppato, permettendo, nei tempi attuali, di eseguire azioni ben più complesse dell'inserimento di semplice testo, come ad esempio impartire il comando di ricevere informazioni meteo, effettuare acquisti, o riprodurre una playlist musicale.

Nel decennio successivo è stato introdotto un nuovo metodo di input in grado di interpretare il linguaggio dei segni, di fatto il precursore del *Gesture Recognition*. Il *Gesture Recognition* non si ferma esclusivamente all'interpretazione e riconoscimento del linguaggio dei segni, ma va oltre, andando ad interpretare gesti e movimenti dell'utente, includendo direzione e velocità dei tali.

Al fine di catturare questi movimenti sono state create diverse tipologie di dispositivi, ognuno con diverse caratteristiche, anche se gran parte di questi sono stati sviluppati per un determinato settore, quello videoludico. La prima azienda videoludica che ha fatto uso di questa innovazione è stata la *Nintendo* con il controller *WiiMote* per la console *Nintendo Wii*, una console molto economica ed accessibile ad un mercato molto ampio, la quale ebbe una diffusione notevole grazie alla novità nel settore.

Dato questo enorme successo, anche altre aziende del settore si cimentarono nel seguire questa nuova modalità di controllo: *Microsoft* sviluppò il sensore *Kinect* per la console *Xbox 360* e successive generazioni, apportando migliorie anche al sensore *Kinect*; la controparte *Sony* sviluppò invece il *Playstation*

Move. Il funzionamento di questi 3 controller ha in comune l'utilizzo di telecamere: il *WiiMote* utilizza una telecamera infrarossi sul telecomando, la quale si orienta verso il monitor grazie ad un dispositivo contenente 10 led infrarossi, che funziona da punto cardine; il *Playstation Move* sfrutta una telecamera in grado di riconoscere le sagome umane, ricreando quindi un modello scheletrico di esso, ed usa dei telecomandi specifici per determinare la persona ed i suoi movimenti; il dispositivo *Kinect*, come spiegato nel capitolo precedente, è invece ben più complesso, motivo per il quale ha trovato largo impiego anche al di fuori del mondo videoludico, venendo utilizzato anche in ambito scientifico grazie alla sua elevata efficienza ed il rilascio da parte di Microsoft del *Software Development Kit (SKD)*, che permette lo sviluppo di software da terze parti sfruttando il suddetto dispositivo.

Negli anni a venire sono nati sempre più dispositivi che permettessero il *Gesture Recognition*, soprattutto nel mondo videoludico ed in particolare per il settore della *Virtual Reality*. I più importanti tra questi sono: *Oculus Rift*, *HTC Vive* e, quello da noi selezionato, il *Leap Motion*.



Figura 3.7: WiiMote[24]



Figura 3.8: Kinect [23]

3.6 Leap Motion

Il nostro sistema fa uso di un controller Leap Motion per il riconoscimento dei movimenti delle mani. Si tratta di un piccolo dispositivo collegabile via USB al PC, che utilizza 2 fotocamere IR monocromatiche, insieme a 3 led infrarossi, per osservare un'area ad una distanza compresa tra 60 e 80 centimetri, con un campo visivo di 140×120 . Nello specifico, il device è in grado di riconoscere tutte le 10 dita con una precisione fino a 1×10^3

3. BACKGROUND

millimetri. Il software del Leap Motion è in grado di riconoscere 27 elementi distinti della mano, incluse ossa e articolazioni, e tracciarle anche nel caso in cui vengano oscurate da altre parti della mano[20]. Il sensore fornisce tre tipi di informazioni spaziali: (1) la posizione di dita e mani; (2) il vettore movimento delle singole dita; (3) la rappresentazione sferica della curvatura della mani. Queste funzionalità permettono all'utente di interagire con il computer attraverso gesture naturali delle mani, come waving, pinching e swiping. L'azienda UltraLeap fornisce delle API in diversi linguaggi di programmazione, e garantisce l'integrazione con Unity3D.

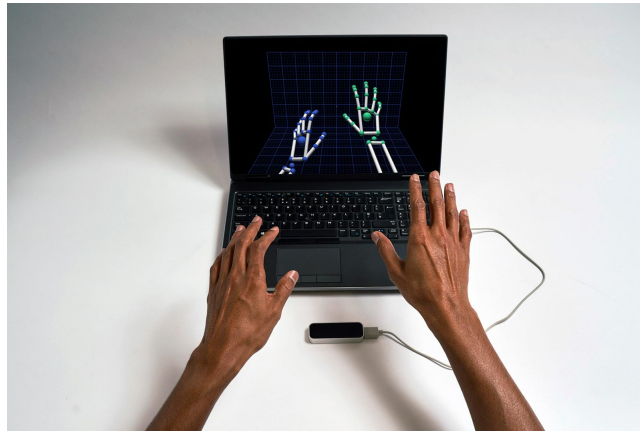


Figura 3.9: Leap Motion Controller

3.7 Artificial Neural Networks

Le *Reti Neurali Artificiali* (*Artificial Neural Networks*, o *ANN*) sono modelli computazionali ispirati alle reti neurali biologiche, come quelle che compongono il cervello, e di conseguenza sono costituiti da un insieme di interconnessioni di informazioni formate tra neuroni artificiali che elaborano dati e li scambiano l'un l'altro, esattamente come le sinapsi. Ogni neurone è connesso con molti altri ed i collegamenti possono essere di tipo rafforzativo o inibitorio rispetto all'attivazione degli altri neuroni ad esso connessi. Ognuno di questi neuroni usa una funzione apposita per combinare tutti i valori di input con una *funzione di attivazione*, che elabora e restituisce l'output del

3. BACKGROUND

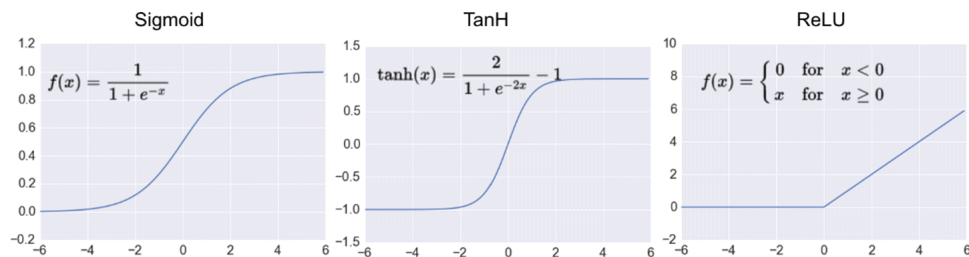
neurone:

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right)$$

Dove w_i è la matrice dei pesi (**weights**) assegnati a ciascun input, che viene aggiunta ad un termine b (**bias**). I due insiemi **weights** e **bias** vengono generati nella fase di addestramento e saranno conservati per l'utilizzo. Tramite questa funzione, la rete neurale filtra gli input, operando solo quelli che rientrano in un determinato limite imposto. Una rete neurale è spesso di tipo *adattivo*, tale da mutare la propria struttura in base a modelli ricevuti in input durante la *fase di apprendimento*.

Per ottimizzare la funzione di attivazione, è possibile introdurre una soglia che servirà a decrementare il valore d'ingresso della funzione. Il metodo più diffuso è l'utilizzo di una delle seguenti funzioni:

- Funzione di **Sigmoid**;
- Funzione di **TanH** (tangente iperbolica);
- Funzione di **ReLU** (rectified linear unit);



La struttura di una rete neurale può essere divisa in tre livelli, ognuno dei quali ha un numero distinto di neuroni:

- Livello di input;
- Livello nascosto;
- Livello di output;

Uno dei difetti delle reti neurali, nonostante l'elevata efficienza, è la non spiegabilità delle decisioni avvenute al loro interno, proprio per questo, questi modelli sono chiamati “*Black Box*”.

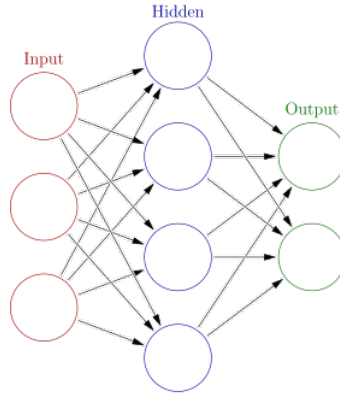


Figura 3.10: Rete Neurale Artificiale

3.8 Paradigmi di apprendimento

L'apprendimento può essere effettuato seguendo tre tipi diversi di paradigmi: ***apprendimento supervisionato***, ***apprendimento non supervisionato*** ed ***apprendimento per rinforzo***.

- **Apprendimento supervisionato (*supervised learning*):**

Quando si ha a disposizione un insieme di dati, anche chiamato *training set*, contenente esempi d'ingresso con il relativo output, la rete imparerà a stabilire la relazione che collega tutti i dati del *training set*.

Nella fase di addestramento, vengono utilizzati appositi algoritmi come il **backpropagation**, che ciclicamente sfrutta i dati ricevuti in input per stimare i *weight* ed i *bias* della rete di neuroni in modo tale da ridurre al minimo l'errore di riconoscimento del *training set*. Se l'addestramento è stato effettuato con successo, quindi con una *accuracy* elevata, la rete ha imparato a riconoscere la relazione che lega i dati in ingresso con quelli in uscita e sarà quindi in grado di effettuare previsioni anche su dati differenti da quelli del *training set*, ovvero su qualsiasi dato (purché rispetti la forma di input accettata) ricevuto in ingresso. La rete sarà

3. BACKGROUND

quindi in grado di risolvere problemi di classificazione e potrà essere chiamata *classifier*.

- **Apprendimento non supervisionato (*unsupervised learning*):**

Metodo di apprendimento basato su algoritmi che riclassificano e organizzano i dati passati in input in base a caratteristiche comuni tra di loro, per effettuare previsioni sugli input ricevuti successivamente. a differenza dell'apprendimento supervisionato, i dati ricevuti in fase di addestramento non sono annotati, dato che le classi da riconoscere non sono conosciute inizialmente, ma verranno distinte durante l'addestramento stesso.

- **Apprendimento per rinforzo (*reinforcement learning*):**

Questo metodo di apprendimento, a differenza dei due precedenti, opera come un automa deterministico, occupandosi di problemi di decisioni sequenziali, dove l'output dipende esclusivamente dallo stato in cui si trova il sistema determinandone il successivo. La qualità delle decisioni dipende da un valore chiamato *ricompensa*, maggiore è la ricompensa e maggiore sarà la propensione del sistema a prendere quella decisione. Le *ricompense* possono essere sia positive che negative, in modo tale da incoraggiare o scoraggiare determinate scelte.

La scelta del paradigma dipende dal tipo di ecosistema che si vuole creare e dal tipo di dati di cui si è in possesso. Se si dispone di dati da fornire in input, e si conosce l'output atteso (detto **label**), la scelta ricade automaticamente sull'*apprendimento supervisionato*.

CAPITOLO 4

MARCOSMILES 2.0

L'obiettivo principale di MarcoSmiles 2.0 è quello di superare i limiti della sua precedente versione: MarcoSmiles [7]. L'idea è quella di permettere all'utente di poter realizzare delle performance musicali in maniera più naturale e performante. Il sistema utilizza esclusivamente informazioni contenute nella configurazione delle mani e associa tali configurazioni a note musicali in real-time. È importante ricordare che il sistema non dispone di una configurazione predefinita. Sarà compito dell'utente creare l'astrazione dello strumento addestrandolo a riconoscere configurazioni delle mani secondo le sue preferenze. Nella sua versione precedente, il lavoro è stato svolto interamente utilizzando il linguaggio di programmazione Java. Durante lo sviluppo di questo lavoro, tuttavia, sono emerse diverse problematiche in merito alla reattività sia dell'interfaccia, che sulle performance in generale, imputabili alla scelta di Java come linguaggio su cui basare il sistema. Inoltre anche la gestione della generazione del suono real-time è risultata particolarmente difficoltosa, motivo per cui, il progetto è stato accantonato. Sulla base di queste esperienze abbiamo deciso di riprogettare il sistema. La scelta sulle tecnologie da usare è ricaduta su Unity3D e C# per quanto riguarda l'interfaccia grafica e le funzioni di base mentre, per la parte di Machine Learning, abbiamo usato

la libreria SciKit Learn in Python. Il sistema da noi sviluppato è dunque strutturato in una componente hardware e una componente software. La componente hardware è rappresentata dal Leap Motion Controller, che può essere posizionato a piacimento dell'utente, solitamente su una scrivania con i sensori rivolti verso l'alto. La componente software consiste nel sistema che effettua l'associazione configurazione - nota musicale in real-time. In fase di progettazione abbiamo suddiviso la parte software in 3 moduli:

1. **Modulo Unity:** Interfacciamento Leap Motion Controller e GUI;
2. **Modulo Machine Learning:** Registrazione dati, elaborazione e testing
3. **Modulo Sonoro:** Generazione audio real-time & Pulse-Code Modulation

Il lavoro di tesi in oggetto riguarda il terzo modulo del sistema. In particolare verrà descritto il motore sonoro del sistema e l'interfaccia grafica collegata ad esso.

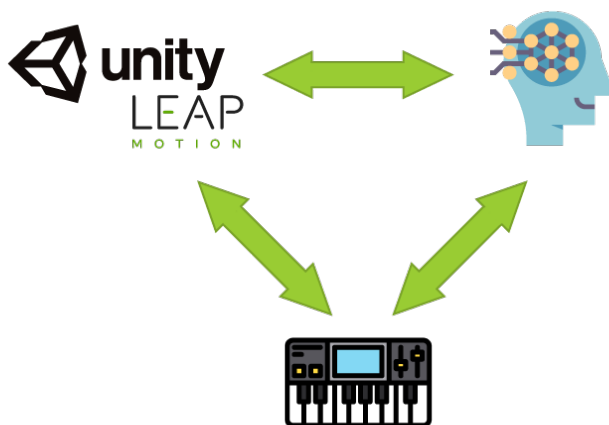


Figura 4.1: Diagramma Interfacciamento Moduli

CAPITOLO 5

MODULO SONORO: L'APPROCCIO UTILIZZATO

Nel seguente capitolo verrà descritto l'approccio proposto nel presente lavoro di tesi. Verranno discusse varie soluzioni implementative e commentate le scelte prese durante la progettazione del modulo sonoro all'interno di MarcoSmiles. In particolare forniremo i dettagli relativi ai seguenti passi: *(i)* Ipotesi implementative, *(ii)* Generazione forme d'onda. Nei capitoli successivi andremo invece a fornire informazioni circa l'implementazione di *(iii)* Effetti & Modulazioni ed *(iiii)* GUI & Oscilloscopio.

5.1 Ipotesi Implementative & Analisi Costi-Benefici

Obiettivi. Durante la progettazione di MarcoSmiles, si sono ipotizzate diverse risoluzioni implementative che risolvessero il problema di generare suoni *real-time*. L'obiettivo principale è stato quello di sviluppare un motore sonoro capace di generare toni in modo reattivo, con il prerequisito di non appesantire il sistema finale e di valorizzare la portabilità sia del motore sonoro stesso, che

del sistema nella propria interezza. Un'altra caratteristica di rilievo alla quale si è fatta attenzione è stata quella di permettere all'utente di personalizzare il suono.

Nel dettaglio, si sono studiate tre ipotetiche soluzioni implementative, illustrate e commentate nelle seguenti sottosezioni. Inoltre, per ognuna di esse è stata fatta un'analisi costi-benefici, con lo scopo di individuare la soluzione ideale per il sistema MarcoSmiles.

5.1.1 Triggers Di Clip Audio

Come spiegato all'interno del Capitolo 3, in Unity una componente AudioSource può riprodurre delle clip audio. La soluzione presentata in questo paragrafo consiste nel riprodurre determinati file audio a seconda della configurazione delle mani assunta dall'utente e dell'output della rete neurale. All'interno di MarcoSmiles, questo concetto sarebbe stato tradotto in pratica nell'immagazzinare all'interno del progetto campionamenti di strumenti reali, in formato WAV o MP3. Dunque avremmo avuto n oggetti di tipo AudioSource all'interno della scena, ognuno contenente una differente clip audio, dove n è il numero di note che possono essere suonate dall'utente.

Analizzando l'ipotesi sono stati individuati i benefici di seguito descritti. Innanzitutto, la soluzione sarebbe stata relativamente semplice da implementare e l'assenza di tecnologie esterne ad Unity avrebbe garantito un semplice funzionamento di essa.

D'altra parte, la soluzione avrebbe limitato significativamente la portabilità dell'applicazione finale, rendendo estremamente difficile estendere il banco di suoni disponibili. Infatti, per aggiungere nuovi suoni nel sistema, sarebbe stato necessario importare n campionamenti di un nuovo strumento, ognuno di una predeterminata lunghezza.

Un altro punto sul quale vale la pena soffermarci è il fatto che problemi di sincronizzazione delle clip audio sarebbero stati altamente probabili. Infatti, nel sistema non è stato previsto alcun modo per fermare un audio in esecuzione, se non quello di togliere le mani dal range di visione del Leap. Dunque, giacché

nel momento in cui l'utente assume una configurazione delle mani, una clip audio viene fatta partire, se l'utente assume diverse configurazioni delle mani in un breve lasso di tempo, più clip audio andrebbero a sovrapporsi l'una all'altra.

Infine, un'ulteriore problematica di significativa importanza è quella che riguarda l'uso delle risorse computazionali, che sarebbe stato più alto rispetto alla soluzione scelta nel sistema finale.

5.1.2 Interfacciamento con la tecnologia MIDI

Questa soluzione consiste nell'interfacciare il nostro sistema con la tecnologia MIDI. Purtroppo, nativamente Unity non presenta alcuna funzione che permetta di usare questa tecnologia all'interno di un progetto. L'idea iniziale è stata quella di utilizzare un VSTPlugin come Serum by XFer o Vital by MattTyet, ma, purtroppo, non è stata trovata alcuna documentazione che supportasse questa idea.

Tuttavia, sono stati esaminati più modi per implementare questa soluzione all'interno del progetto.

Come primo tentativo, è stata studiata la libreria UWP di Windows. In particolare *Windows.Devices.Midi Namespace*, ma, nonostante le numerose ricerche, non si è trovata alcuna documentazione chiara che supportasse questa ipotesi.

Come secondo tentativo è stato studiato un progetto rinominato **CSharpSynthProject** [29], progetto “rispolverato” dalla piattaforma CodePlex (piattaforma di hosting di progetti open source di proprietà Microsoft, in modalità “Archivio” da tre anni , e che sarà chiusa in data 01/07/2021) con ultima modifica datata al 18/07/2014.

Questo progetto contiene un sintetizzatore ideato in C#, che include al suo interno funzionalità MIDI. Supporta la personalizzazione del suono grazie a file di estensione bank e permette di riprodurre delle sequenze midi. Di seguito è riportato un esempio di una classe **MonoBehaviour** che riproduce un file midi usando un determinato timbro descritto da un file bank.

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

Esempio 5.1: Riproduzione file MIDI in un oggetto MonoBehaviour

```
1 using UnityEngine;
2 using System.IO;
3 using System.Collections;
4 using AudioSynthesis;
5 using AudioSynthesis.Bank;
6 using AudioSynthesis.Synthesis;
7 using AudioSynthesis.Sequencer;
8 using AudioSynthesis.Midi;
9
10 namespace UnityMidi
11 {
12     [RequireComponent(typeof(AudioSource))]
13     public class MidiPlayer : MonoBehaviour
14     {
15         [SerializeField] StreamingAssetResource bankSource;
16         [SerializeField] StreamingAssetResource midiSource;
17         [SerializeField] bool loadOnAwake = true;
18         [SerializeField] bool playOnAwake = true;
19         [SerializeField] int channel = 1;
20         [SerializeField] int sampleRate = 44100;
21         [SerializeField] int bufferSize = 1024;
22         PatchBank bank;
23         MidiFile midi;
24         Synthesizer synthesizer;
25         AudioSource audioSource;
26         MidiFileSequencer sequencer;
27         int bufferHead;
28         float[] currentBuffer;
29
30         public AudioSource AudioSource { get { return audioSource; } }
31
32         public MidiFileSequencer Sequencer { get { return sequencer; } }
33
34         public PatchBank Bank { get { return bank; } }
35
36         public MidiFile MidiFile { get { return midi; } }
```

```
37
38     public void Awake()
39     {
40         synthesizer = new Synthesizer(sampleRate, channel, bufferSize, 1);
41         sequencer = new MidiFileSequencer(synthesizer);
42         audioSource = GetComponent();
43
44         if (loadOnAwake)
45         {
46             LoadBank(new PatchBank(bankSource));
47             LoadMidi(new MidiFile(midiSource));
48         }
49
50         if (playOnAwake)
51         {
52             Play();
53         }
54     }
55
56     public void LoadBank(PatchBank bank)
57     {
58         this.bank = bank;
59         synthesizer.UnloadBank();
60         synthesizer.LoadBank(bank);
61     }
62
63     public void LoadMidi(MidiFile midi)
64     {
65         this.midi = midi;
66         sequencer.Stop();
67         sequencer.UnloadMidi();
68         sequencer.LoadMidi(midi);
69     }
70
71     public void Play()
72     {
73         sequencer.Play();
```

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

```
74         audioSource.Play();
75     }
76     /// <summary>
77     /// Writes the raw PCM data, so that it gets played by the AudioSource object
78     /// </summary>
79     /// <param name="data"> Array containing the PCM samples.</param>
80     /// <param name="channel"> number of the channels that are available.
81     /// (for stereo audio channels = 2)</param>
82     void OnAudioFilterRead(float[] data, int channel)
83     {
84         Debug.Assert(this.channel == channel);
85         int count = 0;
86         while (count < data.Length)
87         {
88             if (currentBuffer == null || bufferHead >= currentBuffer.Length)
89             {
90                 sequencer.FillMidiEventQueue();
91                 synthesizer.GetNext();
92                 currentBuffer = synthesizer.WorkingBuffer;
93                 bufferHead = 0;
94             }
95             var length=Mathf.Min(currentBuffer.Length -
96                 bufferHead, data.Length - count);
97             System.Array.Copy(currentBuffer, bufferHead,
98                 data, count, length);
99             bufferHead += length;
100             count += length;
101         }
102     }
103 }
104 }
```

In questo codice viene fatto uso del metodo `MonoBehaviour.OnAudioFilterRead()`.

L'oggetto di tipo *sequencer* contiene una coda dei prossimi eventi midi, mentre l'oggetto *synthesizer*, contiene il buffer *WorkingBuffer* che è un array contenente tutte le informazioni circa il suono che deve essere riprodotto (letteralmente un array di *float*, dove ogni float rappresenta un campionamento PCM). Questo buffer è calcolato sulla base del file bank selezionato e dei prossimi eventi midi contenuti nella coda di eventi nell'oggetto *sequencer*.

Dunque tutto quello di cui avremmo avuto bisogno per riprodurre i suoni all'interno del nostro progetto Unity sarebbe stato questo: leggere gli elementi che si trovano in *synthesizer.WorkingBuffer* e copiarli all'interno dell'array *data[]* usato dalla funzione *OnAudioFilterRead()*.

Questa soluzione sarebbe potuta essere implementata all'interno del nostro sistema, aggiungendo eventi midi all'interno della coda del *sequencer* ad ogni nuova configurazione delle mani assunta dall'utente. Si sarebbe però presentata la rilevante limitazione di poter personalizzare il suono solo tramite l'utilizzo di file ad estensione *bank*. L'utilizzo di questa estensione tuttavia risultava essere ormai in disuso. Risulta infatti essere molto complicato sia ottenere dei file *.bank* nuovi, che recuperare della documentazione chiara che descriva il funzionamento di questi ultimi. Questo è stato uno dei principali punti a sfavore di questa soluzione. Inoltre, seppure il progetto ***CSharpSynthProject*** presentasse delle funzionalità che permettevano di temporizzare gli eventi midi, questa soluzione non avrebbe risolto i problemi di sincronizzazione dovuti a configurazioni delle mani assunte da parte dell'utente in un breve lasso di tempo. Infatti, come già spiegato nella sezione 5.1.1, il sistema non prevede alcun modo per fermare un evento, che, in questo caso, è stato ormai aggiunto al *WorkingBuffer* e che non può dunque, essere fermato.

Questo tentativo ha però illustrato una tecnica che permette di generare toni real-time molto efficace all'interno di Unity, ossia quella di descrivere l'audio che deve essere riprodotto e di farlo eseguire al motore sonoro di Unity,

attraverso l'ausilio del metodo `OnAudioFilterRead()`.

5.1.3 Generazione del suono in modo procedurale utilizzando codifiche PCM

In questo paragrafo viene illustrata la terza ipotesi implementativa. Si comincia, inoltre, a presentare la soluzione definitiva utilizzata all'interno di `MarcoSmiles`.

Nella precedente sezione abbiamo illustrato come sia possibile riprodurre toni in Unity, partendo da una descrizione del segnale. Come spiegato nel capitolo 3, l'array *data* usato dal metodo *OnAudioFilterRead* è un array contenente i campionamenti che rappresentano l'audio che si vuole riprodurre. Dunque, ci basta codificare l'audio che vogliamo riprodurre in campionamenti PCM, per poi copiare i campionamenti all'interno dell'array *data*, in questo modo ci assicuriamo che l'audio codificato venga riprodotto correttamente da un oggetto `AudioSource`.

L'idea di base illustrata in questa soluzione è quella di descrivere funzioni periodiche tramite campionamenti PCM, in modo tale da generare semplici toni.

White noise: Un primo esempio. Questo primo esempio, rappresenta un primo utilizzo del metodo *OnAudioFilterRead*, per riprodurre uno specifico suono, partendo dalla semplice descrizione concettuale di quest'ultimo.

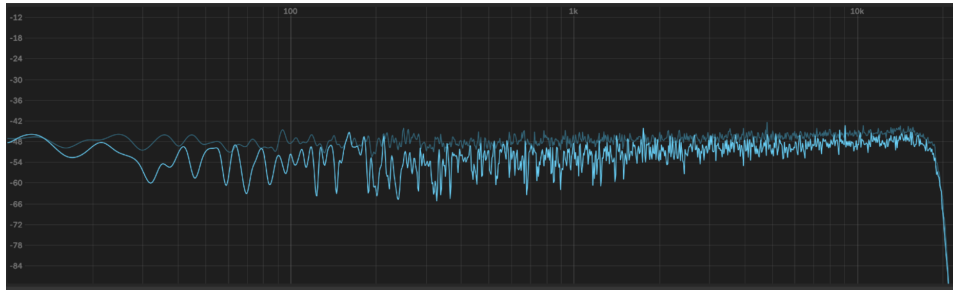


Figura 5.2: Spettrogramma White Noise

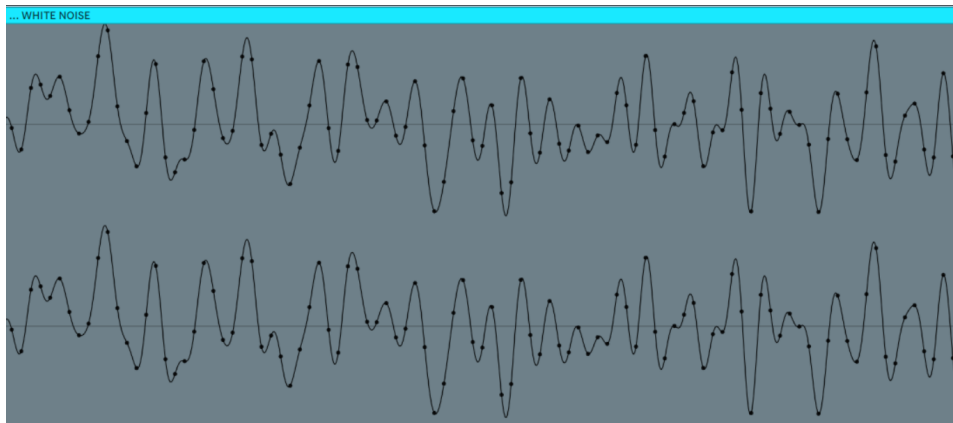


Figura 5.1: Codifica PCM White Noise

Il **White Noise** (rumore bianco) è un particolare tipo di rumore caratterizzato dall'assenza di periodicità nel tempo e da ampiezza costante su tutto lo spettro di frequenze. Dal punto di vista statistico, il rumore bianco è caratterizzato dall'avere valori istantanei del tutto privi di correlazione. Se si campiona un segnale di questo tipo, ciascun valore appare completamente imprevedibile rispetto ai precedenti.

Partendo da questa descrizione, possiamo implementare del codice che riprodurrà white noise all'interno di Unity. Siccome stiamo generando white noise, possiamo ignorare alcuni dettagli come il domandarci quali elementi in *data* riguardano il primo e il secondo canale audio o di conoscere la frequenza di campionamento in uso da Unity. Ci basterà dunque fare uso di *System.Random* per inserire valori random all'interno di ogni campionamento del buffer *data*.

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

Esempio 5.2: Riproduzione White Noise in un oggetto MonoBehaviour

```
1 using UnityEngine;
2 public class WhiteNoise : MonoBehaviour
3 { System.Random rand = new System.Random();
4
5     void OnAudioFilterRead(float[] data, int channels)
6     {
7         for (int i = 0; i < data.Length; i++)
8         {
9             data[i] = (float)(rand.NextDouble() * 2.0 - 1.0);
10        }
11    }
12 }
```

Ogni valore all'interno dell'array *data* deve essere compreso tra -1 e +1. Tutti i valori al di fuori di questo range generano distorsioni e rischiano di *clippare* il segnale. Se si collega lo script ad un GameObject che ha una componente AudioSource al suo interno e si preme su Start, viene generato white noise.

Discutiamo adesso dell'efficacia del metodo appena illustrato, analizzando i benefici e i costi che porterebbe al sistema finale. Innanzitutto, questa soluzione ci permetterebbe di creare toni, senza l'uso di tecnologie o librerie esterne ad Unity. Come secondo punto, la generazione del suono in modo procedurale permetterebbe di creare potenzialmente qualsiasi tono che si desidera, tutto ciò di cui avremo bisogno è la descrizione concettuale e grafica di una forma d'onda ed un algoritmo che la descriva. Inoltre, usando questo metodo, si potrebbero utilizzare modulazioni fra più forme d'onda codificate ed effetti che vanno a modificare il contenuto del buffer *data*, in modo tale da ampliare in maniera significativa il numero di possibili suoni generabili. Consentirebbe, altresì, di mettere a disposizione dell'utente un'interfaccia grafica dedicata alla personalizzazione del suono, dove l'utente può modificare (in fase di esecuzione!) alcuni parametri del suono che si sta generando. Uno dei punti più forti a favore di questa soluzione è il fatto che in questo modo si riuscirebbero a risolvere i problemi di sincronizzazione che hanno caratterizzato le altre due soluzioni illustrate nei precedenti paragrafi. Infatti, in questo

modo possiamo generare suoni che non si fermano mai e che rispondono immediatamente a ciò che sta avvenendo all'interno del sistema. Nella pratica, ad ogni nuova configurazione assunta dall'utente, il suono verrà trasformato per riprodurre una determinata nota; quando le mani vengono spostate al di fuori del range di visione del Leap Motion, il suono si fermerà.

5.1.4 Conclusioni ipotesi illustrate

In questo capitolo abbiamo analizzato vari modi per riprodurre suoni all'interno di Unity. Come è stato anticipato, per la risoluzione del problema della generazione dell'audio real-time è stato scelto di proseguire verso la direzione di una creazione dei suoni in modo procedurale, utilizzando codifiche PCM del segnale. Le motivazioni di questa scelta sono illustrate nei paragrafi precedenti, ma possiamo riassumerle nei seguenti punti:

- Maggiore personalizzazione del suono da parte dell'utente;
- Utilizzo efficiente di risorse computazionali;
- Possibilità di sviluppare ed estendere il codice;
- Maggiore portabilità del codice da un progetto all'altro;
- Risoluzione problematiche di sincronizzazione

Nella prossima sezione illustreremo la possibilità di generare forme d'onda di complessità progressiva, con lo scopo di rendere più chiara la spiegazione della soluzione finale implementata nel sistema.

5.2 Generazione forme d'onda elementari: Un primo approccio

In questa sezione, si descriveranno i vari tentativi, realizzati durante il lavoro di tesi, per riprodurre forme d'onda più o meno complesse. Alla fine di questa sezione, si avranno tutti gli elementi per costruire un vero e proprio sintetizzatore sonoro all'interno di Unity, che possa riprodurre toni di frequenza ed ampiezza variabili e dal timbro personalizzabile.

E' doveroso precisare che i prossimi esempi non presentano il codice definitivo usato all'interno del sistema MarcoSmiles, ma descrivono codice preliminare realizzato durante lo sviluppo del sistema. Questi esempi ci aiuteranno successivamente a spiegare con chiarezza inequivocabile le tecniche utilizzate all'interno dell'implementazione finale.

5.2.1 Sine Wave

Proseguiamo adesso nell'illustrare un esempio di complessità lievemente maggiore (rispetto a quello illustrato nel Par. 5.1.3) .

Un'onda sinusoidale (sine wave) è un'onda descritta matematicamente dalla funzione seno. Una senoide o curva sinusoidale è la curva rappresentata dal grafico del seno.

Questo esempio presenta il caso in cui si voglia generare una forma d'onda sinusoidale all'interno del codice, per poi farla riprodurre dal motore sonoro di Unity.



Figura 5.3: Codifica PCM Sine Wave

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

Esempio 5.3: Riproduzione Sine Wave in un oggetto MonoBehaviour

```
1 using UnityEngine;
2
3 [RequireComponent(typeof(AudioSource))]
4 public class SineWave : MonoBehaviour
5 {
6     // define the frequency of the oscillator
7     double frequency = 440.0;
8     // the volume of the oscillator
9     [Range(0f, 1f)]
10    public float volume = 0.5f;
11
12    // These are needed in order to calculate the oscillation and the samples
13    private double phaseIncrement;
14    private double phase;
15
16    // this represents the next sample of the sine wave
17    private float sinOutput;
18    // this defines the next output sample that needs to be written in the data buffer
19    private float nextOutput;
20
21
22    // Samplerate information
23    private static int sampleRate;
24    //this describes how long should a single sample lasts
25    private static double _sampleDuration;
26    private void Awake()
27    {
28        sampleRate = AudioSettings.outputSampleRate;
29        _sampleDuration = 1.0 / sampleRate;
30    }
31    void OnAudioFilterRead(float[] data, int channels)
32    {
33        for (int i = 0; i < data.Length; i += channels)
34        {
35            /*
36            * The phase variable below increments by the calculated amount for
```

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

```
37      * every audio sample. We can then use this value to calculate what
38      * each waveform's value should be.
39      *
40      *          2pi * f
41      *   phase = -----
42      *          sampleRate
43      *
44      * When phase is greater than 2pi, we just reset
45      * it so we don't have an ever-increasing variable that will cause
46      * an overflow error.
47      */
48
49      phaseIncrement = 2.0 * Mathf.PI * (frequency) / sampleRate;
50      phase += phaseIncrement;
51      if (phase > (Mathf.PI * 2))
52      {
53          phase = phase % (Mathf.PI * 2);
54      }
55
56      // Calculate the next sample for a sine wave
57      sinOutput = (float)(Mathf.Sin((float)phase));
58
59      nextOutput = (float)sinOutput * volume;
60
61      // Writes the output to the audio filter
62      data[i] = nextOutput;
63
64      // Copies the sound from one channel to the other for stereo
65      if (channels == 2)
66          data[i + 1] = data[i];
67  }
68 }
69 }
```

Nel codice di cui sopra, viene fatto uso di concetti elementari di acustica e di audio digitale per i quali si rimanda il lettore al Capitolo 3.

Come prima cosa nel metodo Awake dello script, attraverso il

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

metodo `AudioSettings.outputSampleRate`, otteniamo la frequenza di campionamento in utilizzo dall'output audio di Unity. La variabile `_sampleDuration = 1.0 / sampleRate` descrive la distanza fra un campionamento e l'altro. All'interno del codice, questa variabile non viene utilizzata, ma è importante iniziare a capire cosa rappresenti.

Nel metodo `OnAudioFilterRead` calcoliamo i campionamenti della sinusoide, utilizzando la funzione `Mathf.Sin()` e la fase corrente del segnale. Ad ogni iterazione del ciclo `for`, descriviamo un singolo campionamento. Come prima cosa, calcoliamo l'incremento di fase utilizzando la formula:

$$\text{phaseIncrement} = 2.0 * \text{Mathf.PI} * \text{frequency} / \text{sampleRate}$$

dove *Mathf.PI* è π , *frequency* è la frequenza del segnale e *sampleRate* è la frequenza di campionamento in uso. L'incremento di fase ci permette di calcolare la fase corrente del segnale, conoscendo la fase nell'ultimo campionamento effettuato (iterazione precedente del ciclo `for`). Ad ogni iterazione del ciclo `for`, sommiamo l'incremento di fase appena calcolato alla variabile *phase*. Se *phase* diventa maggiore di 2π , la resettiamo, assicurandoci che la variabile sia in modulo (`Mathf.PI * 2`).

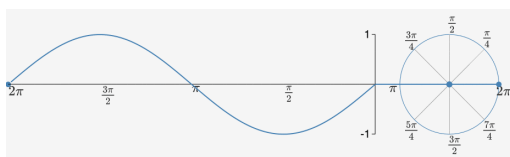


Figura 5.4: Campionamento sine con $\text{phase} = 0$

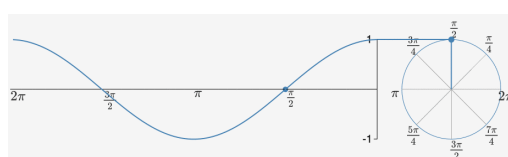


Figura 5.5: Campionamento sine con $\text{phase} = \pi/2$

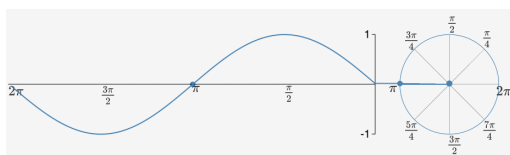


Figura 5.6: Campionamento sine con $\text{phase} = \pi$

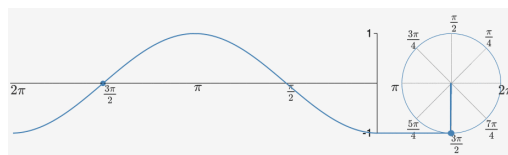


Figura 5.7: Campionamento sine con $\text{phase} = 3\pi/2$

Per ottenere i campionamenti, passiamo l'angolo di fase appena calcolato alla funzione **Mathf.Sin()**, e moltiplichiamo il valore ottenuto per il volume del sintetizzatore. Infine, scriviamo il campionamento effettuato all'interno del buffer **data** e ci assicuriamo di scrivere quest'ultimo in tutte le posizioni dedicate ai canali disponibili (nell'esempio illustrato, i canali disponibili sono due, quindi l'audio sarà stereo).

E' importante capire l'utilità delle variabili **frequency**, **volume** e **nextOutput**. Come già spiegato, **frequency** rappresenta la frequenza in hertz dell'oscillazione del segnale; dunque questa definirà la nota suonata dall'oscillatore. La variabile **volume** è un valore compreso tra 0 ed 1 che viene moltiplicato per il campionamento effettuato e rappresenta il volume del sintetizzatore. Ne concludiamo che possiamo usare queste due variabili per manipolare la nota suonata e il volume del sintetizzatore. La variabile **nextOutput** risulterà invece utile per sommare diverse forme d'onda in un unico output, con l'accortezza di mantenere ad ogni campionamento un valore compreso tra -1 e +1

5.2.2 Square Wave

In questa sezione, spiegheremo come generare un'onda quadra. In teoria dei segnali un'onda quadra (square wave) è un segnale composto da un'alternanza regolare di due valori. I due livelli possono essere messi in corrispondenza con i due stati logici 0 e 1. L'onda quadra perfetta (composta da un'alternanza regolare di due valori) è in realtà un'idealizzazione matematica che nella realtà fisica dei segnali generati da apparati elettronici non esiste a causa del rumore sempre presente in tali apparecchiature.

Nel digitale, possiamo simulare un'onda quadra, alternando ad intervalli regolari due valori, nel nostro caso scegliamo +1 e -1. Possiamo scrivere una classe che riprodurrà una square wave, basandoci sull'esempio presentato nella sezione 5.2.1. Tutto ciò che sarà necessario è sostituire i campionamenti di una sine wave con quelli di una square wave.

Nel codice illustrato successivamente, si è deciso di omettere la parte identica

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

all'esempio precedente e verranno riportati solamente i cambiamenti rispetto a quest'ultimo.

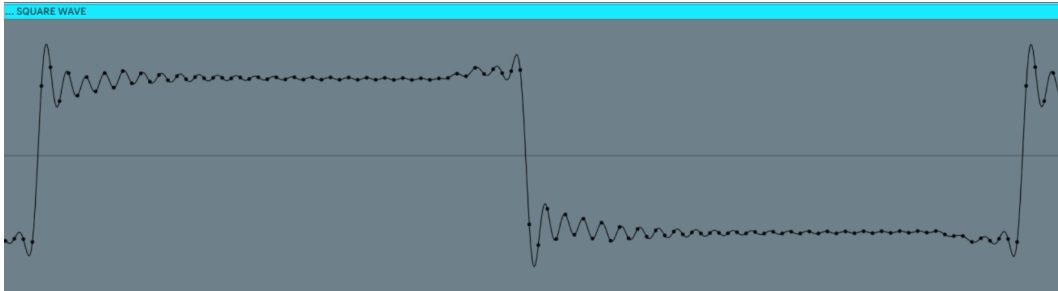


Figura 5.8: Codifica PCM Square Wave

Esempio 5.4: Riproduzione Square Wave in un oggetto MonoBehaviour

```
1 using UnityEngine;
2 [RequireComponent(typeof(AudioSource))]
3 public class SquareWave : MonoBehaviour
4 {
5     ...
6     //this represents the next sample of the square wave
7     private float sqrOutput;
8     ...
9     void OnAudioFilterRead(float[] data, int channels)
10    {
11        for (int i = 0; i < data.Length; i += channels)
12        {
13            ...
14
15            // Calculates the next sample for a square wave
16            if (phase > Mathf.PI)
17                sqrOutput = 1f;
18            else
19                sqrOutput = -1f;
20
21            nextOutput = (float)sqrOutput * volume;
22            data[i] = nextOutput;
23            if (channels == 2)
24                data[i + 1] = data[i];
25        }
26    }
27 }
```

25 }

26

Ancora una volta, come negli esempi precedenti, ci è bastato ragionare sulla descrizione logica del segnale per individuare i campionamenti necessari per descrivere e riprodurre il segnale. In questo caso, abbiamo descritto la square wave nel seguente modo: se la fase attuale del segnale è maggiore di π , allora eseguiamo il campionamento a +1, altrimenti (caso in cui la fase attuale del segnale è minore a π) eseguiamo il campionamento a -1. Prima di scrivere i campionamenti nel buffer *data*, ci assicuriamo di moltiplicare questi per la variabile *volume*, così da controllare il volume in output dall'oscillatore.

$$y(t) = \begin{cases} -1 & \text{se } -\frac{T}{2} \leq t < 0 \\ 1 & \text{se } 0 \leq t \leq \frac{T}{2} \end{cases}$$

5.2.3 Sawtooth Wave

In questa sezione, spiegheremo come generare un'onda a dente di sega (sawtooth wave). La convenzione usuale è che l'onda a dente di sega salga verso l'alto con il passare del tempo e poi scenda bruscamente.

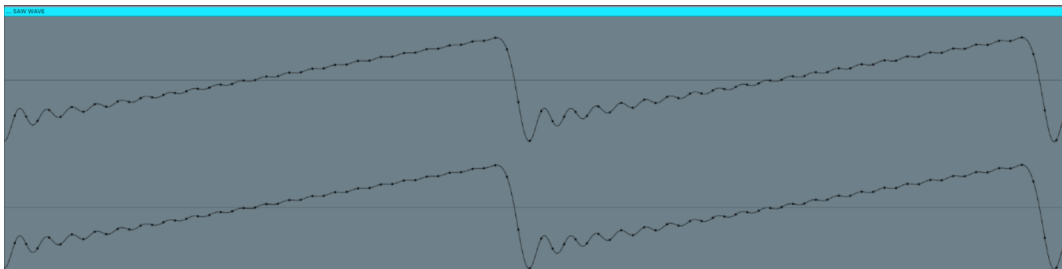


Figura 5.9: Codifica PCM Sawtooth Wave

Esempio 5.5: Riproduzione Saw Wave in un oggetto MonoBehaviour

```
1 using UnityEngine;
2 [RequireComponent(typeof(AudioSource))]
3 public class SawWave : MonoBehaviour
4 {
5     ...
6     //this represents the next sample of the saw wave
7     private float sawOutput;
8     ...
9     void OnAudioFilterRead(float[] data, int channels)
10    {
11        for (int i = 0; i < data.Length; i += channels)
12        { ...
13            // Adds the sample for a sawtooth to the next output sample
14            sawOutput = (float)1 - (float)(1 / Mathf.PI * phase);
15
16            nextOutput = (float)sawOutput * volume;
17            data[i] = nextOutput;
18            if (channels == 2)
19                data[i + 1] = data[i];
20        }
21    }
22 }
```

Possiamo descrivere un campionamento per una sawtooth wave tramite la seguente formula lineare:

$$\text{amplitude} - (\text{amplitude} / \text{Mathf.PI} * \text{phase})$$

dove *Mathf.PI* è π , *amplitude* è l'ampiezza del segnale e *phase* è la fase per la quale si vuole ottenere il campionamento.

All'interno del codice, siccome calcoliamo l'ampiezza del segnale output utilizzando la variabile *volume*, possiamo limitarci a calcolare il campionamento per una sawtooth wave di ampiezza 1. Successivamente ci preoccuperemo di moltiplicare il campionamento per il volume dell'oscillatore.

5.3 Un primo esempio di Sintetizzatore

In questa sezione, uniremo tutte le nozioni spiegate nei precedenti paragrafi per ottenere un primo prototipo di sintetizzatore, che funzioni all'interno di Unity e che sia capace di generare toni personalizzabili.

L'idea è quella di scrivere all'interno del buffer output del sintetizzatore le tre forme d'onda e, per ognuna di esse, controllarne l'ampiezza.

Ricordiamo che attraverso il valore della variabile *frequency* possiamo cambiare la nota suonata dall'oscillatore, mentre la variabile *volume* permette di controllare il volume del sintetizzatore.

Esempio 5.6: Primo esempio di un sintetizzatore all'interno di Unity

```
1 using UnityEngine;
2 [RequireComponent(typeof(AudioSource))]
3 public class Oscillator_v1 : MonoBehaviour
4 {
5     // For oscillation
6     private double phaseIncrement;
7     private double phase;
8     // Samplerate information
9     private static int _FS;
10
11     private float sinOutput;
12     private float sawOutput;
13     private float sqrOutput;
14     private float nextOutput;
15
16     // this controls the amplitude of the general signal
17     [Range(0f, 1f)]
18     public float volume;
19
20     //the frequency of the oscillator, it will define the playing note
21     [Range(20f, 20000f)]
22     public double frequency = 440;
23
24     // Waveshape output weights
```

```
25     [Range(0f, 1f)]
26     public double sinWeight;
27     [Range(0f, 1f)]
28     public double sqrWeight;
29     [Range(0f, 1f)]
30     public double sawWeight;
31
32     void Awake()
33     {
34         //Initial weight for each waveform
35         sinWeight = 0.25;
36         sqrWeight = 0.25;
37         sawWeight = 0.25;
38         // Grabs the sample rate from the system
39         _FS = AudioSettings.outputSampleRate;
40         //Iniziatizes the output variables
41         nextOutput = 0;
42         sinOutput = 0;
43         sawOutput = 0;
44         sqrOutput = 0;
45     }
46
47     void OnAudioFilterRead(float[] data, int channels)
48     {
49         for (int i = 0; i < data.Length; i += channels)
50         {
51             phaseIncrement = frequency * 2.0 * Mathf.PI / _FS;
52             phase += phaseIncrement;
53             if (phase > (Mathf.PI * 2))
54                 phase = phase % (Mathf.PI * 2);
55
56             // Adds sinusoidal wave to the next output sample
57             sinOutput = (float)(sinWeight * Mathf.Sin((float)phase));
58
59             // Adds square wave to the next output sample
60             if (phase > Mathf.PI)
61                 sqrOutput = (float)(sqrWeight);
62             else
```

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

```
62         sqrOutput = (-(float)(sqrWeight));
63
64         // Adds sawtooth wave to the next output sample
65         sawOutput = (float)sawWeight - (float)(sawWeight / Mathf.PI * phase);
66
67         //Combines all the outputs, making sure that nextOutput is never > 1
68         nextOutput = sinOutput + sawOutput + sqrOutput -
69                     (sinOutput * sawOutput) -
70                     (sinOutput * sqrOutput) -
71                     (sawOutput * sqrOutput) +
72                     (sinOutput * sawOutput * sqrOutput);
73
74         nextOutput = nextOutput * volume;
75         data[i] = nextOutput;
76         if (channels == 2) data[i + 1] = data[i];
77     }
78 }
79 }
```



Figura 5.10: Script Oscillator_v1

Ad ogni forma d'onda è assegnata una variabile ***weight***, che descrive il peso che essa avrà sull'output finale. Queste variabili rappresentano in realtà l'ampiezza del segnale che stiamo calcolando. Per modificare il suono generato dal sintetizzatore, modifichiamo il valore delle variabili ***weight***. Inoltre, come negli script precedenti, ad ogni onda è assegnata una variabile ***output***, che servirà a calcolare l'output per quella determinata forma d'onda.

Quindi, in un singolo ciclo for, calcoliamo un campionamento per ogni segnale, sulla base della frequenza (comune a tutte le forme d'onda) e dell'ampiezza (rappresentata dalle variabili ***weight***). Una volta che conosciamo i diversi campionamenti, abbiamo la necessità di combinarli assieme, per ottenere un unico campionamento che andrà poi scritto nel buffer ***data***. La combinazione dei campionamenti è realizzata alla riga 68 dello script *Oscillator_v1*.

Combinare segnali digitali. Nella vita reale, quando si ascoltano suoni che provengono simultaneamente da due fonti diverse, ciò che percepiamo è una somma dei segnali. Se si ascolta un gruppo di dieci persone che cantano contemporaneamente, il risultato sarà più forte di una singola persona che canta. In poche parole, più voci vengono combinate assieme, maggiore sarà l'ampiezza del suono percepito.

Con l'audio digitale, questo non può avvenire. O meglio, questo può avvenire fino a un certo punto. Abbiamo infatti un limitato range di dinamicità. Ad esempio, se usassimo una profondità di campionamento a 8-bit, allora ogni campione sarà un valore compreso tra 0 e 255. Quando combiniamo due segnali del genere, possiamo ottenere un qualsiasi risultato che sia tra 0 e 510 e un campione con ampiezza maggiore a 255, semplicemente “non rientrerebbe” nel range 0-255. Si potrebbe pensare di normalizzare il risultato, dividendolo per due. Questo, tuttavia potrebbe non portare al risultato desiderato. Quando dividiamo, perdiamo informazioni. Nel caso dell'esempio di due segnali ad 8-bit, dividendo per due, riduciamo il range dinamico da 0-255 a 0-127. Questo avviene anche se uno dei due segnali è momentaneamente silente. Se avessimo più di due segnali, questo effetto risulterebbe ancora più palese. Abbiamo

bisogno chiaramente di una tecnica più efficace per risolvere questo problema. Consideriamo due segnali **A** e **B**. Se **A** ha un'ampiezza nulla, vogliamo ascoltare **B** sull'output in una forma inalterata. Viceversa, se **B** ha ampiezza nulla, vogliamo ascoltare **A** in una forma inalterata (**A** e **B** sono trattati in maniera simmetrica). Se sia **A** che **B** hanno un'ampiezza diversa da zero, il segnale combinato deve avere un'ampiezza che sia compresa tra il segnale più intenso, tra **A** e **B**, e il massimo dell'ampiezza permessa. Se prendiamo **A** e **B** con **valori compresi tra 0 e 1**, possiamo usare una semplice equazione:

$$Z = A + B - AB$$

Se aggiungiamo un altro segnale **C** a **Z**, si ottiene:

$$T = Z + C - ZC = A + B + C - AB - AC - BC + ABC$$

Questo è la tecnica utilizzata per combinare i tre output delle tre forme d'onda su un unico output. Ciò viene fatto ovviamente per ogni campionamento.



Figura 5.11: Sezione Oscillatori - MarcoSmiles

Concludiamo questa sezione anticipando come tutto ciò si vada poi a tradurre a livello di interfaccia grafica.

Usando le variabili *public* dello script ***Oscillator_v1***, e tramite il pannello *Inspector* di Unity (*Figura 5.10*), possiamo alterare il segnale in riproduzione, in particolare, possiamo cambiare frequenza, volume e timbro del suono, mantenendo il valore di ogni campionamento in un range adeguato (compreso tra -1 ed +1). Per permettere all'utente di manipolare questi dettagli, prevederemo all'interno dell'applicazione un'interfaccia grafica che andrà a modificare queste variabili e, dunque, ad alterare il suono generato. L'argomento riceverà maggiore approfondimento nel capitolo n. 7 “GUI & Oscilloscopio”.

5.4 Sintesi Additiva: Un nuovo approccio

Nella *Sezione 5.3* siamo riusciti a generare campionamenti di segnali tramite una funzione che calcolasse l'ampiezza della forma d'onda in una precisa fase del segnale; incrementando la fase in maniera costante otteniamo i campionamenti. In questa sezione, cambiamo totalmente approccio di sintesi, saremo così in grado di generare forme d'onda complesse, semplicemente sommando sinusoidi fra loro. Questo tipo di sintesi è chiamata **sintesi additiva** ed è la tecnica utilizzata all'interno del sistema MarcoSmiles.

La sintesi additiva è basata sul teorema di Fourier.

Teorema di Fourier. Il teorema di Fourier afferma che qualunque funzione periodica di periodo T_0 o, ugualmente, di frequenza $f_0 = 1/T_0$, continua e limitata può essere rappresentata mediante una somma di funzioni sinusoidali pure di opportuna ampiezza e di frequenza multipla della frequenza fondamentale f_0 .

Segnali sinusoidali che siano multipli di una frequenza fondamentale f_0 sono chiamati **armoniche naturali** di quella frequenza e corrispondono alle frequenze naturali delle armoniche di una corda vibrante. Se ordiniamo dalla più grave alla più acuta queste frequenze otteniamo la scala delle armoniche naturali. Questa "scala" non è stata creata o decisa dall'uomo, ma risponde ad una legge di natura, è la conseguenza di un fenomeno fisico e come tale può essere spiegato in termini matematici. Il modello matematico che descrive questo schema è chiamato **serie armonica**. Il modello matematico della serie armonica è piuttosto semplice: data una frequenza fondamentale f , la serie armonica da essa generata è la successione dei suoi multipli interi $(f * 1, f * 2, f * 3 \dots)$. Per semplicità queste frequenze multiple vengono dette **armoniche** o armonici.

La trasformata di Fourier veloce (FFT) di un suono ci permette di analizzare la serie armonica di quel suono e dunque l'ampiezza di ogni armonica. Questa rappresentazione viene spesso chiamata spettro della funzione.

Spettrogramma. Si tratta di una rappresentazione grafica su un diagramma cartesiano che presenta in ascissa le frequenze ed in ordinata una grandezza rappresentativa dell'ampiezza del suono. Uno spettrogramma si ottiene, di solito, suddividendo l'intervallo di tempo totale (cioè quello relativo all'intera forma d'onda da analizzare) in sotto-intervalli uguali (detti finestre temporali) di durata da 5 a 10 ms e calcolando la trasformata di Fourier della parte di forma d'onda contenuta in ciascuna finestra (solitamente si usa la trasformata veloce di Fourier, o FFT), che fornisce l'intensità del suono in funzione della frequenza. Le trasformate di Fourier, relative alle diverse finestre temporali, vengono poi assemblate a formare lo spettrogramma.

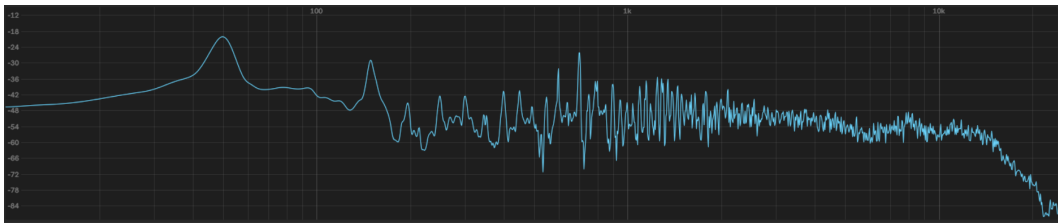


Figura 5.12: Spettrogramma di un suono complesso

Nelle prossime sezioni, riprenderemo lo studio fatto in precedenza, applicando i concetti appena illustrati, con lo scopo di descrivere forme d'onda complesse usando la sintesi additiva.

Il codice presentato, rappresenta il codice utilizzato all'interno del nostro sistema.

5.4.1 La classe *SinusWave*

Procederemo nel descrivere la classe *SinusWave*. Il funzionamento di questa classe è piuttosto simile alle tecniche utilizzate nelle sezioni 5.2 e 5.3. Nonostante ciò, il codice contiene dei sostanziali miglioramenti.

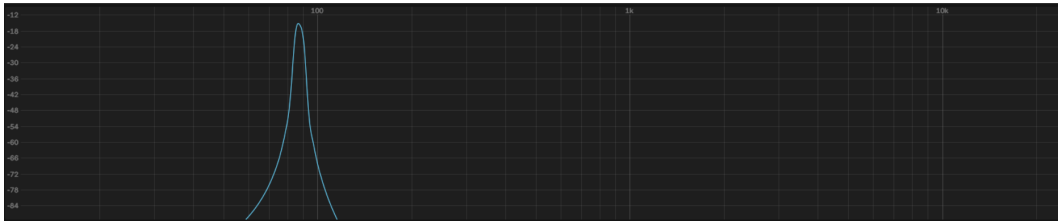


Figura 5.13: Spettrogramma sine wave con frequenza = 87 Hz

Esempio 5.7: Implementazione Classe SinusWave.cs

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4 using System;
5
6 public class SinusWave
7 {
8     double currentSignalTime;           // the current dsp time
9     double currentSignalFrequency;       // the current frequency of the signal
10    double currentSignalPhaseOffset;     // the current phase offset of the signal
11
12    double currentSignalOutlValue;       //the current signal output value
13
14    public SinusWave()
15    {
16        currentSignalFrequency = 400.0;
17        currentSignalPhaseOffset = 0.0;
18    }
19
20    public double calculateSignalValue(double newSignalTime, double newSignalFrequency)
21    {
22
23        if (currentSignalFrequency != newSignalFrequency)
24        {
25            // calculate the signal's current period: period = 1 / frequency
26            double currentSignalPeriod = 1.0 / currentSignalFrequency;
27            /*calculate the current number of cycles:
28            the number of cycles is the number of times that a complete period
```

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

```
29 of the wave has occured.*/
30     double currentNumberOfSignalCycles = (currentSignalTime / currentSignalPeriod) +
31         (currentSignalPhaseOffset / (2.0 * Math.PI));
32 //current cycle position
33     double currentSignalCyclePosition = currentNumberOfSignalCycles % 1.0;
34
35     double newSignalPeriod = 1.0 / newSignalFrequency;
36     double newNumberOfSignalCycles = currentSignalTime / newSignalPeriod;
37
38 //new cycle position
39     double newSignalCyclePosition = newNumberOfSignalCycles % 1.0;
40
41     double cycleDifference = currentSignalCyclePosition - newSignalCyclePosition;
42     double phaseDifference = cycleDifference * Math.PI * 2.0;
43
44     currentSignalPhaseOffset = phaseDifference;
45
46     currentSignalFrequency = newSignalFrequency;
47     currentSignalTime = newSignalTime;
48
49     currentSignalOutlValue = Math.Sin(currentSignalTime * 2.0 * Math.PI *
50         currentSignalFrequency + currentSignalPhaseOffset);
51     return currentSignalOutlValue;
52 }
53 else
54 {
55     currentSignalFrequency = newSignalFrequency;
56     currentSignalTime = newSignalTime;
57     currentSignalOutlValue = Math.Sin(currentSignalTime * 2.0 * Math.PI *
58         currentSignalFrequency + currentSignalPhaseOffset);
59     return currentSignalOutlValue;
60 }
61 }
62 }
```

Il metodo *calculateSignalValue()* riceve come parametri *newSignalTime* e *newSignalFrequency*. *newSignalTime* rappresenta il tempo in cui vogliamo ottenere il valore del campionamento, nella pratica questo sarà il **tempo dsp** (Digital Signal Processing Time) del motore audio di Unity. Possiamo accedere al **tempo dsp** tramite l'oggetto `AudioSettings` messo a disposizione dalle API di Unity, ed in particolare utilizzando ***AudioSettings.dspTime***. Come descritto all'interno della documentazione di Unity, il **tempo dsp** è un valore in secondi basato sul numero di campionamenti processati dal motore audio.

newSignalFrequency rappresenta invece la frequenza del segnale che vogliamo campionare.

Passando il **tempo dsp** e la **frequenza del segnale** come parametri attuali al metodo *calculateSignalValue()* otterremo il campionamento.

Il codice che va dalla riga 23 alla riga 44 nell'*Esempio 5.7*, rappresenta il primo miglioramento qualitativo rispetto al codice della **Sezione 5.2**. Questa parte di codice gestisce i cambiamenti di frequenza del segnale (quando *newSignalFrequency* è diversa da *currentSignalFrequency*), in modo tale da evitare di scrivere rumori e “click” nei campionamenti. Questi rumori sono causati dal fatto che quando la frequenza del segnale cambia, la forma d'onda che si sta generando cambia improvvisamente la sua fase.

Per rendere tutto più chiaro, supponiamo che si verifichi un cambiamento da $\text{Sin}(2 * \text{PI} * 0.75) = -1$ a $\text{Sin}(2 * \text{PI} * 0.8) = -0.951056516295$. Il cambio improvviso da -1 a -0.951056516295, causa una discontinuità nel grafico della funzione sinusoidale, e questo può generare dei “clicks” sull'output audio.

Il modo in cui questo problema è stato risolto all'interno del codice consiste nello *shiftare* la fase del segnale in modo tale che il valore della funzione sinusoidale rimane identico fra i due step (prima e dopo il cambio di frequenza). In questo modo, indipendentemente da quanto velocemente avvengono i cambiamenti di frequenza, non verranno mai riprodotti “clicks” e rumori dovuti a questa circostanza.

Attraverso la classe **SinusWave** e la **sintesi additiva** saremo in grado di

riprodurre qualsiasi forma d'onda.

5.4.2 La classe SquareWave

Finalmente, analizzeremo una forma d'onda generata tramite **sintesi additiva**. Iniziamo individuando una semplice analisi della serie armonica per una square wave.

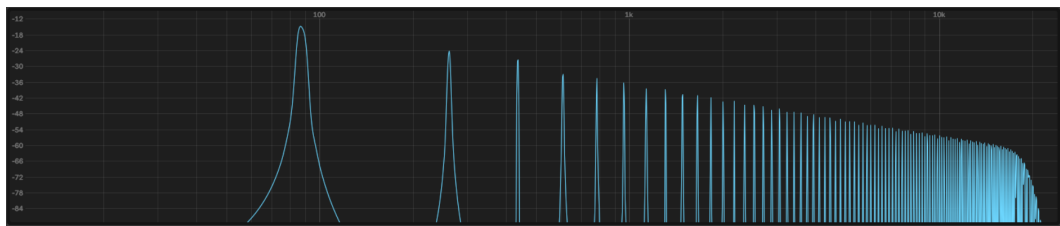


Figura 5.14: Spettrogramma square wave con frequenza fondamentale = 87 Hz

Serie armonica Square Wave. La serie armonica di una square wave (fonda d'onda quadra) è costituita da una senoide fondamentale (che definirà la frequenza della square wave) e da altri segnali sinusoidali, multipli dispari della frequenza fondamentale (**armoniche dispari**). L'ampiezza delle armoniche è uguale a $1/N$, dove N è l'armonica (1, 3, 5, 7...).

Se iniziamo ad aggiungere una prima armonica dispari (in verde nella Figura 5.15) di ampiezza di circa $1/3$ rispetto all'ampiezza della frequenza fondamentale ad una senoide, che sarà la nostra frequenza fondamentale, (in rosso nella Figura 5.15) possiamo osservare come il segnale risultante (in nero nella Figura 5.15) inizia ad assomigliare a quello di una square wave.

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

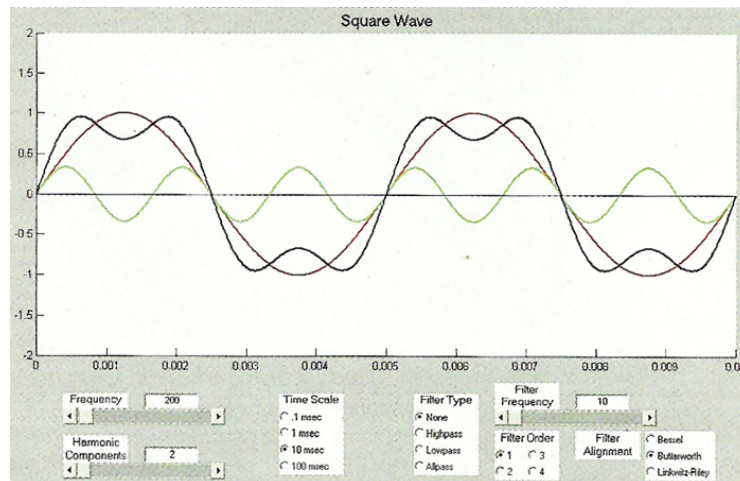


Figura 5.15: Segnale generato dalle prime due armoniche di una square wave

Se aumentiamo il numero di armoniche dispari aggiunte, da 2 a 5, otteniamo un segnale molto simile a quello di una square wave (Figura 5.16).

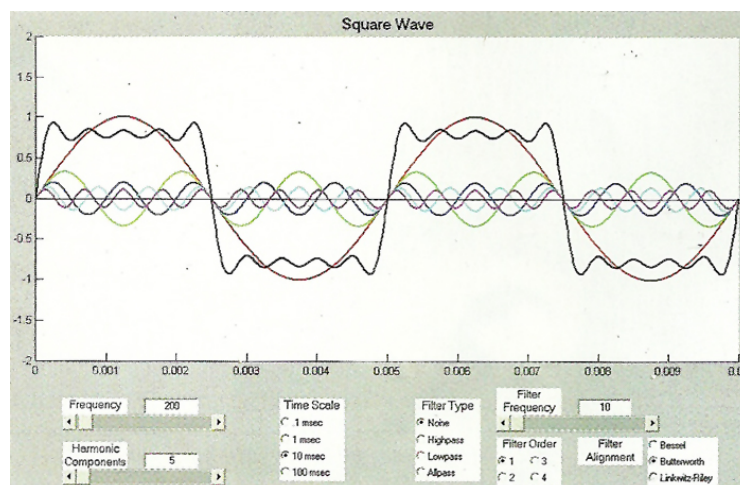


Figura 5.16: Segnale generato dalle prime cinque armoniche dispari

Per ottenere una square wave, ci basta dunque aggiungere un numero sufficiente di armoniche dispari ad una frequenza fondamentale.

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

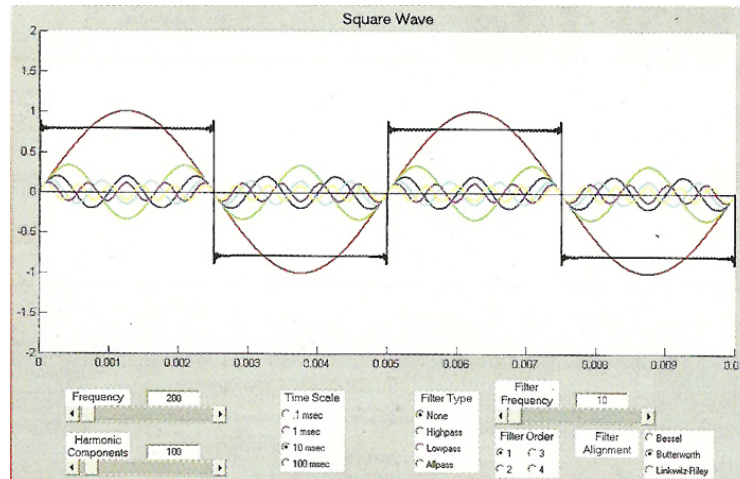


Figura 5.17: Segnale generato dalle prime dieci armoniche dispari

Con queste conoscenze preliminari, siamo pronti a scrivere la nostra classe **SquareWave** che ci permetterà di avere i campionamenti di un'onda a forma quadra, generata tramite **sintesi additiva**.

Esempio 5.8: Implementazione Classe SquareWave.cs

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4 using System;
5
6 public class SquareWave
7 {
8     double[] squareHarmonicLevels = new double[]
9     {
10         1.0,    // level of 1st harmonic
11         0.0,    // level of 2nd harmonic
12         0.33333333, // level of 3rd harmonic
13         0.0,    // level of 4th harmonic
14         0.2,    // level of 5th harmonic
15         0.0,    // level of 6th harmonic
16         0.14285714, // level of 7th harmonic
17         0.0,    // level of 8th harmonic
18         0.11111111, // level of 9th harmonic
19         0.0,    // level of 10th harmonic
```

```
20     0.09090909, // level of 11th harmonic
21     0.0,        // level of 12th harmonic
22     0.07692308, // level of 13th harmonic
23     0.0,        // level of 14th harmonic
24     0.06666666, // level of 15th harmonic
25     0.0         // level of 16th harmonic
26 };
27
28 SinusWave[] subSignals;
29
30 public SquareWave()
31 {
32     subSignals = new SinusWave[16];
33     for (int i = 0; i < 16; i++)
34     {
35         subSignals[i] = new SinusWave();
36     }
37 }
38
39 public double calculateSignalValue(double newSignalTime, double newSignalFrequency)
40 {
41     double signalSum = 0.0;
42     for (int i = 0; i < subSignals.Length; i++)
43     {
44         signalSum += squareHarmonicLevels[i] *
45             subSignals[i].calculateSignalValue(newSignalTime, newSignalFrequency * (i + 1));
46     }
47     return signalSum;
48 }
49 }
```

La classe **SquareWave** descrive un'approssimazione di una square wave, utilizzando **armoniche dispari**. Le armoniche sono generate attraverso l'ausilio della classe **SinusWave**, illustrata nella sezione precedente. L'array *squareHarmonicLevels* conterrà l'ampiezza relativa alle prime sedici armoniche del segnale ossia $1/N$, dove N è l'armonica (1,3,5,7...). Siccome per approssimare una square wave abbiamo bisogno solo delle armoniche dispari,

tutte le armoniche pari avranno intensità pari a 0. Per ottenere i campionamenti, sommiamo i campionamenti di tutte le armoniche usando un ciclo for (riga 42) e, per ottenere la frequenza di ogni armonica, moltiplichiamo la frequenza della fondamentale per la posizione dell'armonica all'interno della serie armonica.

In questo modo possiamo generare una square wave che sia più "liscia" di una square wave assoluta. Questo ci permetterà di ridurre alcuni rumori che una square wave assoluta potrebbe generare.

5.4.3 La classe SawWave

In campo audio, il suono di un'onda a dente di sega, all'orecchio risulta duro e chiaro, il suo spettro contiene armoniche sia pari sia dispari della frequenza fondamentale.

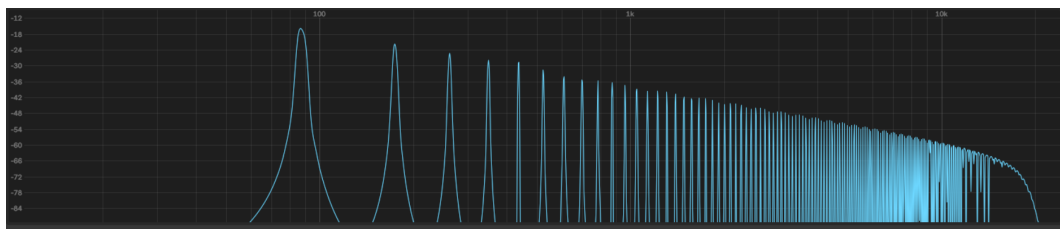


Figura 5.18: Spettrogramma saw wave con frequenza fondamentale = 87 Hz

Serie armonica Saw Wave. La serie armonica di una saw wave (fonda d'onda a dente di sega) è costituita da una sinusoide fondamentale (che definirà la frequenza della saw wave) e da tutte le armoniche intere. L'ampiezza delle armoniche è uguale a $1/N$, dove N è l'armonica (1, 2, 3, 4, 5, 6...).

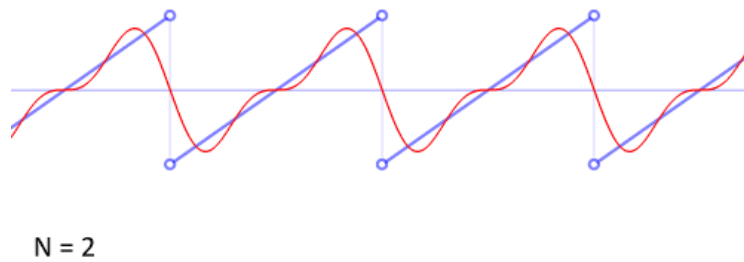


Figura 5.19: Segnale generato dalle prime due armoniche di una saw wave

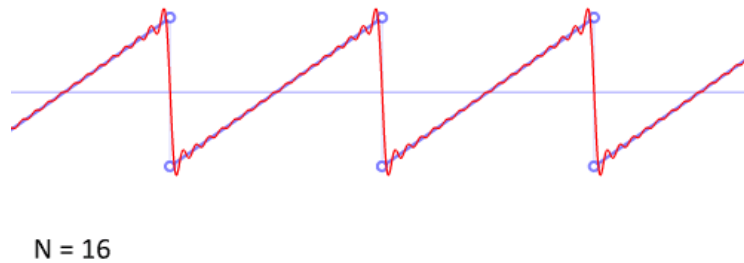


Figura 5.20: Segnale generato da 16 armoniche intere

Esempio 5.9: Implementazione Classe SawWave.cs

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4 using System;
5
6 public class SawWave
7 {
8     double[] sawHarmonicLevels = new double[]
9     {
10         1.0,    // level of 1st harmonic
11         0.5,    // level of 2nd harmonic
12         0.33333333, // level of 3rd harmonic
13         0.25,   // level of 4th harmonic
14         0.2,    // level of 5th harmonic
```

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

```
15     0.16666666,    // level of 6th harmonic
16     0.14285714,    // level of 7th harmonic
17     0.125,         // level of 8th harmonic
18     0.11111111,    // level of 9th harmonic
19     0.1,           // level of 10th harmonic
20     0.09090909,    // level of 11th harmonic
21     0.08333333,    // level of 12th harmonic
22     0.07692308,    // level of 13th harmonic
23     0.07142857,    // level of 14th harmonic
24     0.06666666,    // level of 15th harmonic
25     0.0625         // level of 16th harmonic
26 };
27
28 SinusWave[] subSignals;
29
30 public SawWave()
31 {
32     subSignals = new SinusWave[16];
33     for (int i = 0; i < 16; i++)
34     {
35         subSignals[i] = new SinusWave();
36     }
37 }
38
39 public double calculateSignalValue(double newSignalTime, double newSignalFrequency)
40 {
41     double signalSum = 0.0;
42     for (int i = 0; i < subSignals.Length; i++)
43     {
44         signalSum += sawHarmonicLevels[i] *
45             subSignals[i].calculateSignalValue(newSignalTime, newSignalFrequency * (i + 1));
46     }
47     return signalSum;
48 }
49 }
```

La classe **SawWave** descrive un'approssimazione di una saw wave, utilizzando **armoniche intere**. Così come per la classe **SquareWave**, le armoniche sono generate attraverso l'ausilio della classe **SinusWave**. L'array *squareHarmonicLevels* conterrà l'ampiezza relativa alle prime sedici armoniche del segnale ossia $1/N$, dove N è l'armonica (1, 2, 3, 4, 5, 6...). Come fatto in precedenza, per ottenere i campionamenti, sommiamo i campionamenti di tutte le armoniche usando un ciclo for e, per ottenere la frequenza di ogni armonica, moltiplichiamo la frequenza della fondamentale per la posizione dell'armonica all'interno della serie armonica.

Anche in questo caso, andremo a generare una saw wave più "liscia" rispetto a una saw wave assoluta e dunque andremo a ridurre alcuni rumori che una saw wave assoluta andrebbe a generare.

5.5 Sintetizzatore in MarcoSmiles

Abbiamo adesso le tre classi **SinusWave.cs**, **SquareWave.cs** e **SawWave.cs**, che ci permetteranno di descrivere le tre corrispettive forme d'onda.

Possiamo perfezionare il codice presentato nella Sezione 5.3, utilizzando le nuove classi create. Illustreremo per chiarezza prima un esempio esplicativo e successivamente il codice utilizzato all'interno del sintetizzatore nel sistema finale.

Esempio 5.10: Nuova versione Sintetizzatore

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4 [RequireComponent(typeof(AudioSource))]
5 public class Oscillator_v2 : MonoBehaviour
6 {
7     SawWave sawAudioWave;
8     SquareWave squareAudioWave;
```

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

```
9      SinusWave sinusAudioWave;
10
11      private int sampleRate; // samples per second
12      private double dataLen; // the data length of each channel
13      double chunkTime;
14      double dspTimeStep;
15      double currentDspTime;
16
17      private float nextOutput;
18
19      // Waveshape output weights
20      [Range(0f, 1f)]
21      public double sinWeight;
22      [Range(0f, 1f)]
23      public double sqrWeight;
24      [Range(0f, 1f)]
25      public double sawWeight;
26
27      //waves actual output
28      private float sinOutput;
29      private float sawOutput;
30      private float sqrOutput;
31
32      //general volume of the oscillators, the output is multiplied by this value
33      [Range(0f, 1f)]
34      public float volume = 0.5f;
35
36
37      // Manages Frequencies and notes
38      private float[] frequencies = new float[] {
39      261.630f , 277.180f , 293.660f , 311.130f , 329.630f , 349.990f ,
40      369.990f , 392.000f , 415.300f , 440.000f , 466.160f , 493.880f };
41
42      private float octave = 1f;
43
44      [Range(0,11)]
45      public int freqIndex;
```

```
46
47 void Awake()
48 { sawAudioWave = new SawWave();
49   squareAudioWave = new SquareWave();
50   sinusAudioWave = new SinusWave();
51
52   sampleRate = AudioSettings.outputSampleRate;
53 }
54
55 void Start()
56 {   sinWeight = 0.75; sqrWeight = 0.25; sawWeight = 0.25;
57   nextOutput = 0; sinOutput = 0; sawOutput = 0; sqrOutput = 0;
58 }
59
60 void OnAudioFilterRead(float[] data, int channels)
61 {
62   /*
63   * This is "the current time of the audio system", as given
64   * by Unity. It is updated every time the OnAudioFilterRead() function
65   * is called. It's usually every 1024 samples.
66   */
67   currentDspTime = AudioSettings.dspTime;
68   // the actual data length for each channel
69   dataLen = data.Length / channels;
70   // the time that each chunk of data lasts
71   chunkTime = dataLen / sampleRate;
72   // the time of each dsp step.(the time that each individual audio sample lasts)
73   dspTimeStep = chunkTime / dataLen;
74
75   double preciseDspTime;
76
77   for (int i = 0; i < dataLen; i++) // go through the data buffer
78   {
79       preciseDspTime = currentDspTime + i * dspTimeStep;
80
81       float currentFreq = frequencies[freqIndex];
82
```

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

```
83 sinOutput = (float)
84     (sinWeight * sinusAudioWave.calculateSignalValue(preciseDspTime, currentFreq));
85
86 sawOutput = (float)
87     (sawWeight * sawAudioWave.calculateSignalValue(preciseDspTime, currentFreq));
88
89 sqrOutput = (float)
90     (sqrWeight * squareAudioWave.calculateSignalValue(preciseDspTime, currentFreq));
91
92     //Combines all the outputs, making sure that nextOutput is never > 1
93     nextOutput = sinOutput + sawOutput + sqrOutput -
94                 (sinOutput * sawOutput) -
95                 (sinOutput * sqrOutput) -
96                 (sawOutput * sqrOutput) +
97                 (sinOutput * sawOutput * sqrOutput);
98
99     float x = volume * (float)nextOutput;
100    //writes the sample on the available channels
101    for (int j = 0; j < channels; j++)
102    {
103        data[i * channels + j] = x;
104    }
105 }
106 }
107 }
```

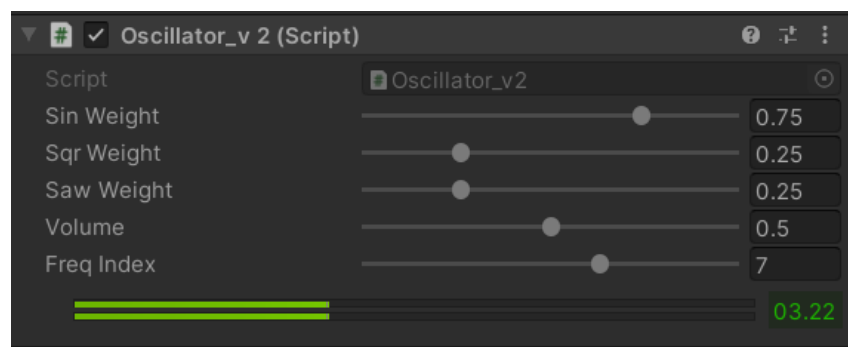


Figura 5.21: Script Oscillator_v2.cs

Come nella versione precedente illustrata nella Sezione 5.3, possiamo

manipolare l'onda generata, modificando il peso delle tre forme d'onda sull'output, e controllare il volume del sintetizzatore. In questo script è previsto un metodo più raffinato per cambiare la nota (e di conseguenza la frequenza) riprodotta dal sintetizzatore. Questo metodo consiste nell'avere un array contenente le frequenze di un'ottava di note (nel nostro caso l'ottava che parte da DO4) ed un indice che rappresenta quale nota stia suonando l'oscillatore. La variabile *octave* non è stata usata nel codice, ma potrebbe eventualmente essere usata per cambiare l'ottava di partenza dell'array *frequencies*. Per farlo, ci basterà moltiplicare la variabile *octave* per 1/2 per spostarci in un'ottava più bassa, per 2 per spostarci in un'ottava più alta. Successivamente procediamo nel moltiplicare tutti gli elementi dell'array *frequencies* per la variabile *octave*. In questo modo possiamo cambiare l'ottava di note riproducibili.

Per calcolare i campionamenti delle singole forme d'onda, avremo bisogno del **tempo dsp** (Digital Signal Processing Time) del motore audio di Unity. Per far sì che all'interno del nostro codice questo tempo sia il più preciso possibile (e dunque per eseguire campionamenti che siano il più preciso possibile), calcoliamo la durata di un singolo campionamento audio (*dspTimeStep*). Ad ogni iterazione del primo ciclo for, sommeremo *dspTimeStep* a *currentDspTime* in modo tale da effettuare campionamenti ad intervalli regolari (per avere un incremento di fase costante). Il primo ciclo for, verrà ripetuto tante volte quanto è la grandezza del buffer *data* per un singolo canale. Alla fine di ogni iterazione copiamo il campionamento appena effettuato sugli altri canali disponibili.

Utilizziamo i metodi *calculateSignalValue* per ottenere i campionamenti per ogni forma d'onda, successivamente combiniamo tutti i campionamenti nella variabile *nextOutput* (usando il metodo illustrato nella Sezione 5.3), che rappresenta il campionamento da scrivere nel buffer.

5.5.1 Implementazione Finale & Interfacciamento con gli altri Moduli

Finalmente, abbiamo tutti gli elementi per illustrare il codice implementato nel sistema finale e spiegare come vengono riprodotte le note all'interno di **MarcoSmiles**. E' doveroso precisare che, nel codice seguente, troveremo anche concetti ai quali non abbiamo ancora fatto cenno, ma che verranno successivamente spiegati nel Capitolo 6 "Modulazioni & Effetti".

Esempio 5.11: ProceduralAudioOscillator.cs

```
1 [RequireComponent(typeof(AudioSource))]
2 public class ProceduralAudioOscillator : MonoBehaviour
3 {
4     SawWave sawAudioWave;
5     SquareWave squareAudioWave;
6     SinusWave sinusAudioWave;
7     //For amplitude and frequency modulation
8     SinusWave amplitudeModulationOscillator;
9     SinusWave frequencyModulationOscillator;
10    public double frequency = 440;
11    private int sampleRate; // samples per second
12    private double dataLen; // the data length of each channel
13    double chunkTime;
14    double dspTimeStep;
15    double currentDspTime;
16    // Waveshape output weights
17    [Range(0f, 1f)]
18    public double sinWeight;
19    [Range(0f, 1f)]
20    public double sqrWeight;
21    [Range(0f, 1f)]
22    public double sawWeight;
23    private float sinOutput; private float sawOutput; private float sqrOutput;
24    private float nextOutput;
25    private float previousOutput; // For single-pole low pass filter
26
27    [Header("Amplitude Modulation")]
```

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

```
28     public bool useAmplitudeModulation;
29     //Float parameter that determines the Amplitude Modulation Oscillators frequency.
30     [Range(0.2f, 30.0f)]
31     public float amplitudeModulationOscillatorFrequency = 1.0f;
32
33 [Header("Frequency Modulation")]
34     public bool useFrequencyModulation;
35     //Float parameter that determines the Frequency Modulation Oscillators frequency.
36     [Range(0.2f, 30.0f)]
37     public float frequencyModulationOscillatorFrequency = 1.0f;
38     //Float parameter that determines the Frequency Modulation Oscillators intensity.
39     [Range(1.0f, 100.0f)]
40     public float frequencyModulationOscillatorIntensity = 10.0f;
41     /*
42     These parameters do not control the Synthesizer, but can be used to
43     drive (control) other scripts parameters.
44     */
45 [Header("Out Values")]
46     //The Amplitude Modulation Oscillators current value (range 0 to 1)
47     [Range(0.0f, 1.0f)]
48     public float amplitudeModulationRangeOut;
49     //The Frequency Modulation Oscillators current value (range 0 to 1)
50     [Range(0.0f, 1.0f)]
51     public float frequencyModulationRangeOut;
52
53     /* These control the amplitude of the general signal */
54     //volume of the oscillators, the output is multiplied by this value
55     [Range(0f, 1f)]
56     public float volume = 0;
57     //the value that is assigned to the variable volume
58     [Range(0f, 1f)]
59     public float volumeValue;
60     // Manage Frequencies and notes
61     private float[] frequencies = new float[] {
62         261.630f , 277.180f , 293.660f , 311.130f , 329.630f , 349.990f ,
63         369.990f , 392.000f , 415.300f , 440.000f , 466.160f , 493.880f };
64     /* These control the single pole low pass filter */
```

```
65     [Range(0, 1)]
66     public double lowPass;
67
68
69     void Awake()
70     {
71         sawAudioWave = new SawWave();
72         squareAudioWave = new SquareWave();
73         sinusAudioWave = new SinusWave();
74         amplitudeModulationOscillator = new SinusWave();
75         frequencyModulationOscillator = new SinusWave();
76         sampleRate = AudioSettings.outputSampleRate;
77
78         //the value that is assigned to the variable volume, this is the actual volume of the
79         volumeValue = 0.3f;
80         volume = volumeValue;
81     }
82
83     void Start()
84     {
85         sinWeight = 0.75;  sqrWeight = 0.25; sawWeight = 0.25;
86         lowPass = 1;
87         changeNote(_GM.indexPlayingNote);
88     }
89
90     void Update()
91     {
92         changeNote(_GM.indexPlayingNote);
93         //Debug.Log(frequency);
94     }
95
96     void changeNote(int noteIndex)
97     { /* Uses a switch case....
98      * changes the frequency of the oscillator to match the frequency
99      * of the note represented by noteIndex */
100     }
101
```

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

```
102 void OnAudioFilterRead(float[] data, int channels)
103 {
104     if (_GM.isActive)
105     {
106         currentDspTime = AudioSettings.dspTime;
107         dataLen = data.Length / channels;
108         chunkTime = dataLen / sampleRate;
109         dspTimeStep = chunkTime / dataLen;
110         double preciseDspTime;
111
112         for (int i = 0; i < dataLen; i++)
113         {
114             preciseDspTime = currentDspTime + i * dspTimeStep;
115             double currentFreq = frequency;
116             //Applies Frequency Modulation
117             if (useFrequencyModulation)
118             {
119                 double freqOffset =
120                     (frequencyModulationOscillatorIntensity * frequency * 0.75) / 100.0;
121                 currentFreq +=
122                     mapValueD(frequencyModulationOscillator.calculateSignalValue(
123                         preciseDspTime, frequencyModulationOscillatorFrequency),
124                         -1.0, 1.0, -freqOffset, freqOffset);
125                 frequencyModulationRangeOut =
126                     (float)frequencyModulationOscillator.calculateSignalValue(
127                         preciseDspTime, frequencyModulationOscillatorFrequency) * 0.5f + 0.5f;
128             }
129             else
130             { frequencyModulationRangeOut = 0.0f; }
131
132             sinOutput = (float)
133                 (sinWeight * sinusAudioWave.calculateSignalValue(preciseDspTime, currentFreq));
134             sawOutput = (float)
135                 (sawWeight * sawAudioWave.calculateSignalValue(preciseDspTime, currentFreq));
136             sqrOutput = (float)
137                 (sqrWeight * squareAudioWave.calculateSignalValue(preciseDspTime, currentFreq));
138
```

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

```
139         //Combines all the outputs, making sure that nextOutput is never > 1
140         nextOutput = sinOutput + sawOutput + sqrOutput -
141             (sinOutput * sawOutput) -
142             (sinOutput * sqrOutput) -
143             (sawOutput * sqrOutput) +
144             (sinOutput * sawOutput * sqrOutput);
145 //Applies Amplitude Modulation
146         if (useAmplitudeModulation)
147         {
148             nextOutput *=
149                 (float)mapValueD(amplitudeModulationOscillator.calculateSignalValue(
150                     preciseDspTime, amplitudeModulationOscillatorFrequency), -1.0, 1.0, 0.0, 1.0);
151             amplitudeModulationRangeOut =
152                 (float)amplitudeModulationOscillator.calculateSignalValue(
153                     preciseDspTime, amplitudeModulationOscillatorFrequency) * 0.5f + 0.5f;
154         }
155         else
156         { amplitudeModulationRangeOut = 0.0f; }
157
158         float x = volume * (float)nextOutput;
159
160         for (int j = 0; j < channels; j++)
161         {
162             data[i * channels + j] = x;
163         }
164     }
165 }
166 }
167 public void OctaveUp()
168 {
169     for (int i = 0; i < frequencies.Length; i++)
170         frequencies[i] *= 2;
171 }
172 public void OctaveDown()
173 {
174     for (int i = 0; i < frequencies.Length; i++)
175         frequencies[i] /= 2;
```

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

```
176     }
177     /* This function maps (converts) a Double value from one range to another */
178     double mapValueD(double referenceValue, double fromMin,
179                     double fromMax, double toMin, double toMax)
180     {
181         return
182             toMin + (referenceValue - fromMin) * (toMax - toMin) / (fromMax - fromMin);
183     }
184 }
```

5. MODULO SONORO: L'APPROCCIO UTILIZZATO

La classe `_GM`, rappresenta il “Game Master” del nostro sistema. E’ una classe *MonoBehaviour* che mantiene aggiornato lo stato dell’applicazione. Attraverso `_GM`, lo script **ProceduralAudioOscillator.cs** si interfaccia con gli altri due moduli del sistema. Lo fa utilizzando sostanzialmente due variabili. La prima è `_GM.indexPlayingNote` che rappresenta l’indice della nota da suonare. Quest’indice è calcolato dalla rete neurale a partire dalla configurazione delle mani assunta dall’utente. L’indice `_GM.indexPlayingNote` è mantenuto aggiornato all’interno di `_GM`, e verrà aggiornato ad ogni nuova configurazione delle mani assunta dall’utente (in realtà, viene aggiornata dal metodo `Update` di `_GM`). Osservando l’indice della nota da suonare, **ProceduralAudioOscillator.cs** è in grado di cambiare la frequenza dell’oscillatore per suonare quella determinata nota. E lo fa tramite il metodo *changeNote()*, che non è altro che uno switch case che cambia la frequenza del synth a seconda dell’indice che gli viene passato. In poche parole, possiamo dire che la variabile *indexPlayingNote* permette di interfacciare il modulo sonoro con il modulo machine learning del sistema. La seconda variabile è `_GM.isActive`. Questa variabile descrive se le mani sono correntemente rilevate dal LeapMotion. Se lo sono, *isActive* sarà uguale a *true*; altrimenti *isActive* sarà uguale a *false*. Nel sistema, non vogliamo emettere alcun suono quando le mani non sono rilevate. Dunque, prima di scrivere campionamenti all’interno del buffer *data*, ci basterà controllare che *isActive* sia uguale a *true*.

Come anticipato nella sezione precedente, in questo codice abbiamo previsto, inoltre, la possibilità di spostare l’ottava di note di partenza che si sta suonando. Facciamo questo tramite i metodi *OctaveUp()* ed *OctaveDown()* che moltiplicheranno rispettivamente l’array *frequencies* per 2 e per 1/2.

CAPITOLO 6

MODULAZIONI & EFFETTI

In questo capitolo verrà spiegato come è possibile, attraverso l'utilizzo della funzione ***OnAudioFilterRead()***, modulare il segnale generato ed aggiungere effetti sull'output del *synth*.

Nel codice illustrato nell'**Esempio 5.11** è stato incluso il codice sia per modulare la frequenza dell'oscillatore, usando la frequenza di una nuova sinusoide (***frequencyModulationOscillator***), che il codice per modulare l'ampiezza del segnale dell'oscillatore, usando la frequenza di un'ennesima sinusoide (***amplitudeModulationOscillator***). Andiamo ad analizzare nel dettaglio come queste modulazioni vengono applicate sul segnale.

6.1 Frequency Modulation

La modulazione di frequenza, sigla **FM** (dall'analogo termine inglese frequency modulation), è una delle tecniche di trasmissione utilizzate per trasmettere informazioni usando la variazione di frequenza dell'onda portante [33]. In musica e nella riproduzione audio la sintesi a modulazione di frequenza, o **sintesi FM**, è una tecnica di sintesi sonora grazie alla quale il timbro di una forma d'onda semplice viene modificato modulando la sua frequenza con una

frequenza di modulazione, ottenendo una forma d'onda più complessa ed un tono con un suono differente.

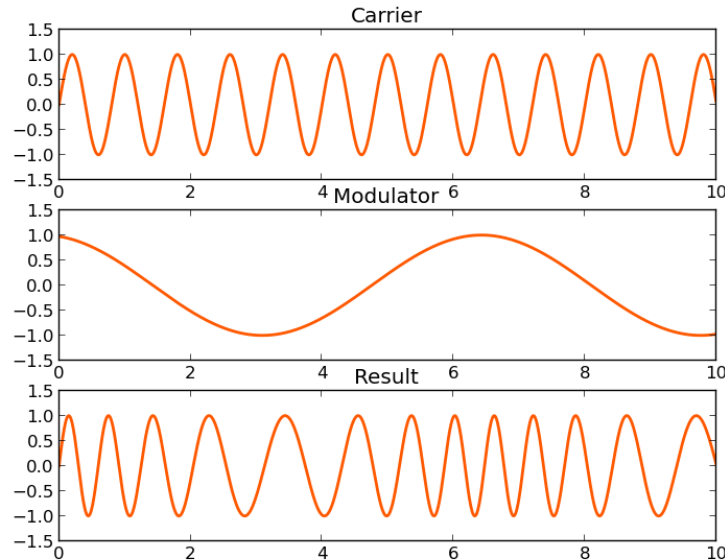


Figura 6.1: Sintesi FM

Per poter effettuare frequency modulation sul segnale del sintetizzatore, avremo dunque bisogno di modulare la frequenza stessa dell'oscillatore, utilizzando un qualsiasi altro segnale. Nell'Esempio 5.11, abbiamo scelto di utilizzare una sine wave come segnale modulante. Possiamo usare la classe **SinusWave** per definire il segnale modulante, che chiameremo ***frequencyModulationOscillator***. Oltre questa variabile, avremo bisogno di descrivere la frequenza del segnale modulante (***frequencyModulationOscillatorFrequency***) e l'intensità della modulazione che stiamo applicando (***frequencyModulationOscillatorIntensity***). Avendo questi dati, possiamo procedere modificando la frequenza utilizzata dall'oscillatore per calcolare i campionamenti delle forme d'onda. ***frequencyModulationRangeOut*** è una variabile che rappresenta la modulazione in corso e che potrebbe dunque essere usata per modificare altri parametri.

6.2 Amplitude Modulation

La modulazione di ampiezza, sigla AM (dall'analogo termine inglese *amplitude modulation*), è una tecnica di trasmissione usata per trasmettere informazioni utilizzando un segnale come segnale portante. Consiste nel modulare l'ampiezza di un segnale in maniera proporzionale all'ampiezza di un secondo segnale modulante e che contiene informazione [34]. Utilizzeremo questa tecnica per modulare l'ampiezza dell'output del *synth* con l'ampiezza di una sinusoide (ma potremo usare tranquillamente qualsiasi altra forma d'onda).

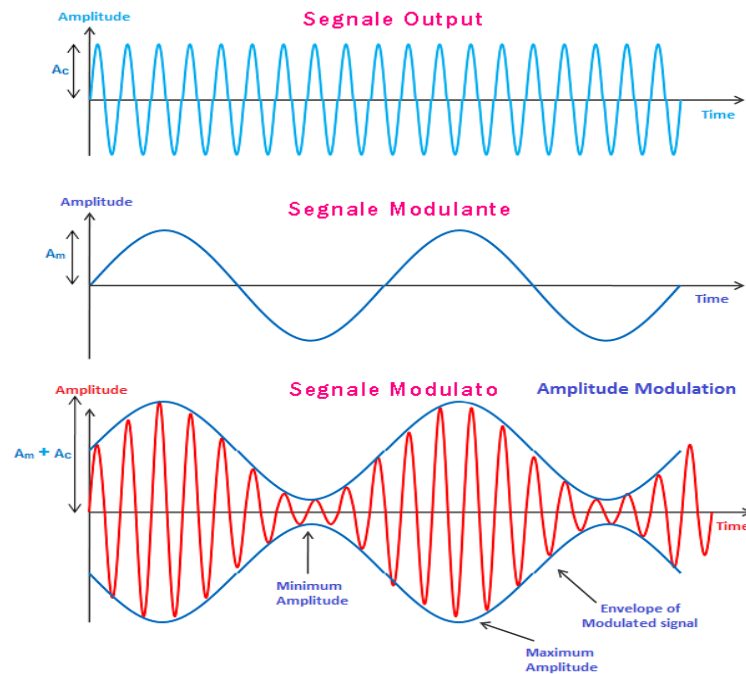


Figura 6.2: Modulazione AM

Per poter effettuare amplitude modulation sul segnale del sintetizzatore, avremo dunque bisogno di modulare l'ampiezza stessa del segnale output dell'oscillatore, utilizzando un qualsiasi altro segnale. Nell'Esempio 5.11, abbiamo scelto di utilizzare una sine wave come segnale modulante. Chiameremo il segnale modulante *amplitudeModulationOscillator*. Oltre questa variabile, avremo bisogno di descrivere la frequenza del segnale modulante (*amplitudeModulationOscillatorFrequency*). Avendo questi

dati, possiamo procedere modificando l'ampiezza dei campionamenti effettuati, prima di scriverli nel buffer *data*. *amplitudeModulationRangeOut* è una variabile che rappresenta la modulazione in corso e che potrebbe dunque essere usata per modificare altri parametri.



Figura 6.3: Componenti Effettistica Synth - MarcoSmiles

6.3 Riverbero, Chorus & Filtro Passa Basso

Per l'implementazione degli effetti Riverbero, Chorus e Filtro Passa Basso, sono state usate componenti **Audio Filter** di Unity: rispettivamente **Audio Reverb Filter**, **Audio Chorus Filter** e **Audio Low Pass Filter**. Per ogni componente sono stati scelti alcuni parametri da collegare alle manopole del synth, in modo tale da permettere la modifica degli effetti da parte dell'utente.

CAPITOLO 7

GUI & OSCILLOSCOPIO

All'interno di MarcoSmiles, è stata implementata un'interfaccia grafica per permettere all'utente di modificare parametri relativi al suono che si sta generando (**Figura 7.1**). Il design è totalmente ispirato a quello di un sintetizzatore reale, infatti sono previste delle manopole, che simulano i potenziometri di un *synth* fisico.

Per analizzare il suono che si sta generando, è stato implementato un oscilloscopio che disegnerà la forma d'onda che il sintetizzatore e Unity stanno riproducendo.

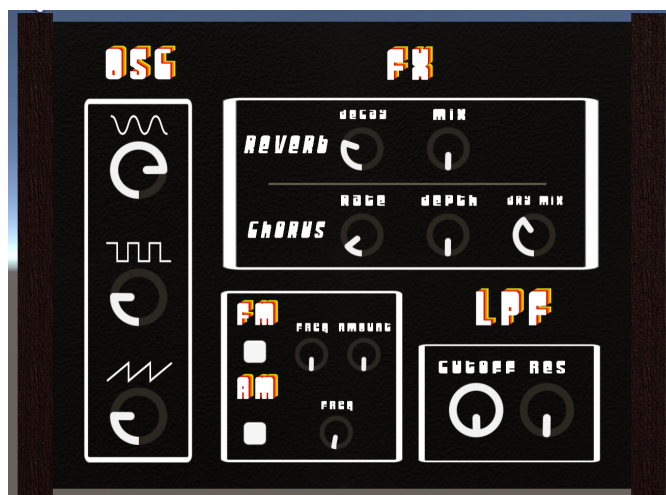


Figura 7.1: Sintetizzatore - MarcoSmiles

7.1 GUI Sintetizzatore

Come spiegato in precedenza, nell'interfaccia grafica sono state implementate delle manopole, che servono a modificare parametri del sintetizzatore ed a modificare di conseguenza il suono generato da esso. Per l'implementazione grafica di queste, è stata utilizzata una repository open source trovata su GitHub intitolata **VJUI** e realizzata da **Keijiro Takahashi**.

VJUI. VJUI [35] è una collezione di Unity Assets che rappresentano controlli, ideati per il *VJing*. Contiene tre oggetti personalizzabili: oggetto Knob; oggetto Button; oggetto Toggle. L'oggetto Knob permette all'utente di selezionare un valore numerico, trascinando l'oggetto e ruotandolo. Questo prevede una funzionalità simile all'oggetto Slider di Unity, con la sostanziale differenza consistente nel fatto che quest'ultimo ha una forma rotonda.

Tra gli oggetti presenti in VJUI, utilizzeremo solamente l'oggetto **Knob**, che permetterà all'utente di controllare i vari parametri del synth. Per ogni oggetto **Knob**, possiamo collegare un metodo. Ogni volta che il valore di un oggetto **Knob** verrà cambiato, verrà passato il valore attuale dell'oggetto al metodo collegato. In questo modo riusciamo a cambiare qualsiasi parametro dello script **ProceduralAudioOscillator.cs** attraverso l'interfaccia grafica del sintetizzatore.

Un'importante caratteristica dell'oggetto **Knob** è quella di poter definire il range di valori che esso può assumere (e dunque di evitare di passare valori troppo grandi o troppo piccoli al metodo collegato all'oggetto).

Per attivare e disattivare le modulazioni sul segnale, sono stati invece utilizzati oggetti **Toggle** di Unity.

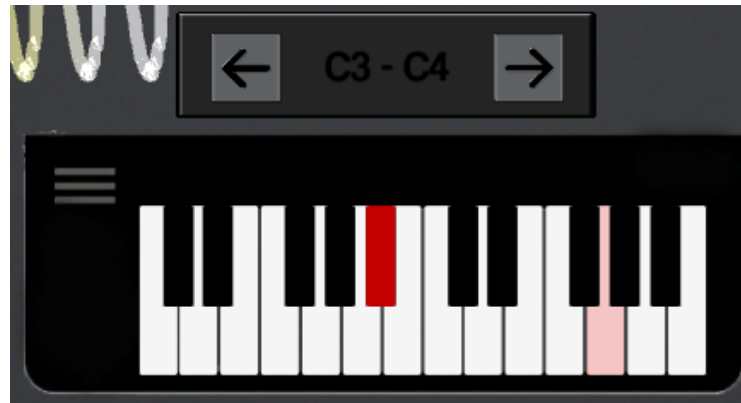


Figura 7.2: Tasti per cambiare ottava - MarcoSmiles

E' stata implementata anche un'interfaccia grafica per permettere all'utente di cambiare la nota di partenza delle ottave suonabili, collegando i metodi *OctaveUp()* ed *OctaveDown()* mostrati nell'Esempio 5.11 ai tasti in **Figura 7.2**

7.2 Oscilloscopio

L'oscilloscopio è uno strumento di misura elettronico che consente di visualizzare, su un grafico bidimensionale, l'andamento nel dominio del tempo dei segnali elettrici ed effettuare misure a lettura diretta di ampiezza (rappresentata sull'asse verticale) e periodo (con il tempo rappresentato sull'asse orizzontale).

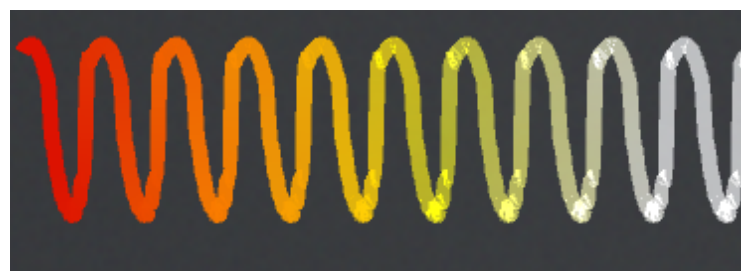


Figura 7.3: Oscilloscopio - MarcoSmiles

La realizzazione dell'oscilloscopio nel nostro sistema si basa sullo script **Oscilloscope.cs** illustrato di seguito.

7. GUI & OSCILLOSCOPIO

Esempio 7.1: Oscilloscope.cs

```
1 using UnityEngine;
2 using UnityEngine.UI;
3 [DisallowMultipleComponent]
4 public class Oscilloscope : MonoBehaviour
5 {
6     [SerializeField] private Modes mode;
7     private static int samples = 1024;
8     private LineRenderer lineRenderer;
9     public AudioSource audioSource;
10
11     private void Awake()
12     {
13         lineRenderer = GetComponent<LineRenderer>();
14         //audioSource = GetComponent<AudioSource>();
15     }
16     private void Update()
17     {
18         float[] samplesL = new float[samples];
19         float[] samplesR = new float[samples];
20
21         audioSource.GetOutputData(samplesL, 0);
22         audioSource.GetOutputData(samplesR, 1);
23
24         Vector3[] positions = new Vector3[samples];
25         Vector3[] old_positions = new Vector3[samples];
26         lineRenderer.GetPositions(old_positions);
27
28         for (int i = 0; i < samples; i++){
29             positions[i] = new Vector3((float)i /
30                 (samples - 1) * 2 - 1, (samplesL[i] + samplesR[i]) / 2);
31         }
32         lineRenderer.positionCount = samples;
33         lineRenderer.SetPositions(positions);
34     }
35 }
```

7. GUI & OSCILLOSCOPIO

Con questo semplice script, andiamo a “disegnare” il contenuto del buffer ***data***, attraverso l'utilizzo di una componente **LineRenderer**. E' necessario collegare l'oggetto **AudioSource** del quale vogliamo leggere il buffer ***data*** per far funzionare lo script illustrato. La variabile ***samples*** è il numero di campionamenti che andiamo a disegnare e rappresenta la risoluzione dell'oscilloscopio.

CAPITOLO 8

CONCLUSIONI

In questo capitolo conclusivo discuteremo circa gli obiettivi che ci eravamo riproposti di raggiungere e di quelli raggiunti effettivamente durante il lavoro. Commenteremo inoltre alcuni dettagli all'interno del sistema MarcoSmiles e del modulo sonoro realizzato.

8.1 Risultati e Discussioni

Lo scopo principale del lavoro è stato quello di progettare e realizzare un'applicazione che permettesse di suonare uno strumento virtuale utilizzando solamente un sensore Leap Motion ed i movimenti delle mani. In fase di progettazione sono stati individuati dal team di sviluppo tre moduli diversi: Modulo Unity & Leap Motion; Modulo Machine Learning e Modulo Sonoro. In particolare, questo lavoro di tesi si è concentrato nello sviluppo di un sintetizzatore audio che fosse capace di generare toni *real-time*, totalmente personalizzabili da parte dell'utente. La scelta implementativa di utilizzare Unity3D ci ha supportato nella creazione dell'applicazione, garantendo interoperabilità fra i vari moduli e fornendo strumenti eccellenti per descrivere un'interfaccia grafica efficace. Ha però implicato delle limitazioni per ciò che

8. CONCLUSIONI

concerne il modulo sonoro, in quanto l'uso del linguaggio C# e l'estrema difficoltà nell'importare librerie esterne all'interno del game engine hanno causato alcune difficoltà nello sviluppo del modulo. Sono state analizzate, dunque, più soluzioni implementative e tra queste la tecnica più efficace è stata quella di utilizzare il metodo `MonoBehaviour.OnAudioFilterRead()` per inoltrare campionamenti di audio digitale al motore sonoro di Unity. Attraverso algoritmi che descrivono codifiche PCM, siamo riusciti a far riprodurre dal motore sonoro di Unity determinate forme d'onda. Inoltre, scalando gli output generati, siamo riusciti a combinare più segnali assieme, senza causare problemi di *clipping* del segnale. Successivamente sono state studiate nuove tecniche di sintesi del suono, con l'obiettivo di migliorarne il timbro ed aumentarne la possibile personalizzazione da parte dell'utente. La ricerca ci ha portato ad usare un approccio diverso, che ci ha consentito di generare codifiche PCM di segnali complessi attraverso la tecnica di sintesi additiva. Dunque, abbiamo costruito una classe `SinusWave`, che descrive i campionamenti PCM per un segnale sinusoidale e successivamente abbiamo costruito le classi `SquareWave` e `SawWave`, che ci hanno permesso di costruire forme d'onda quadre e forme d'onda a dente di sega. Questo è fatto descrivendo la serie armonica di queste onde e, dunque, (per il teorema di Fourier) usando solamente una somma di segnali sinusoidali. In questo modo siamo riusciti a costruire uno script che contiene tutto il funzionamento di un sintetizzatore digitale ed a generare forme d'onda più lisce che non generano rumori o click. Per allargare le possibilità di personalizzazione del suono, abbiamo implementato la possibilità di applicare modulazioni sul segnale generato dal sintetizzatore, in particolare modulazione di frequenza e di ampiezza. Inoltre abbiamo inserito altri effetti quali riverbero, chorus e filtro passa basso. Infine, il lavoro si è concentrato sull'interfaccia grafica, con lo scopo di rendere agevole la modifica dei parametri del sintetizzatore e di conseguenza la personalizzazione del suono riprodotto. All'interno dell'interfaccia grafica è stato previsto anche un oscilloscopio, che permette di analizzare e visualizzare la forma d'onda generata.

8.2 Lavori futuri

In questa sezione verranno discussi alcuni lavori futuri che potrebbero estendere il sistema MarcoSmiles, per ciò che concerne il modulo sonoro.

- *Aumento espressività del suono*: nelle versioni future, sarà possibile aumentare l'espressività del suono, modificando parametri del sintetizzatore attraverso specifici movimenti delle mani. Ad esempio, si potrebbe aumentare e diminuire il volume avvicinando ed allontanando le mani dal sensore Leap Motion, oppure utilizzare una mano per suonare note ed un'altra per modificare parametri.
- *Estensione forme d'onda disponibili*: basandosi sul concetto di sintesi additiva, si potranno descrivere forme d'onda che riproducano un timbro che sia simile a strumenti reali, descrivendo lo spettro sonoro di questi ultimi.
- *Aggiunta presets*: si potranno mettere a disposizione dei suoni pre-fabbricati (presets). Questo potrà essere fatto salvando i parametri del sintetizzatore.
- *Estensione Modulazioni*: si potrebbe mettere a disposizione dell'utente un'interfaccia che permetta di scegliere quale tipo di forma d'onda usare per modulare il segnale output del synth. Si potrebbero addirittura creare algoritmi di routing della modulazione, prevedendo la possibilità che ogni oscillatore di forma d'onda possa modulare un altro oscillatore. Questo permetterebbe di estendere la possibilità di personalizzazione del suono in maniera esponenziale.
- *Interfacciamento con MIDI e DAW*: in futuro, il sistema potrebbe essere sviluppato per assomigliare di più ad un VstPlugin che utilizzi la tecnologia MIDI in maniera efficace e che si interfacci con Digital Audio Workstation quali Ableton Live o Steinberg Cubase. In questo modo, il sistema potrebbe essere usato come strumento midi per registrare sequenze MIDI all'interno di DAW.

BIBLIOGRAFIA

- [1] Maxine Sherrin, August de los Reyes: Predicting the past Web Directions, 2008
- [2] Ditte Hvas Mortensen, Natural User Interfaces – What are they and how do you design user interfaces that feel natural Interaction Design Foundation, 2020,
- [3] Computer Music - https://en.wikipedia.org/wiki/Computer_music
- [4] Pietro Grossi Computer Music - <http://www.hackerart.org/corsi/aba02/taddepera/computer>
- [5] Ariana Grande Sings Harmonies with Mimu Gloves, San Jose April 12th, 2015 - <https://www.youtube.com/watch?v=sa6h6wZpE30>
- [6] Mi.Mu Gloves - <https://mimugloves.com/>
- [7] Natural User Interfaces to Support and Enhance Real-Time Music Performance - Rocco Zaccagnino, Roberto De Prisco, Delfina Malandrino, Gianluca Zaccagnino
- [8] Accessible Digital Musical Instruments — A Review of Musical Interfaces in Inclusive Music Practice, Emma Frid KTH Royal Institute of Technology, Lindstedtsvägen 3, 100 44 Stockholm, Sweden
- [9] Theremin - <https://it.wikipedia.org/wiki/Theremin>

BIBLIOGRAFIA

- [10] Mellotron - <https://it.wikipedia.org/wiki/Mellotron>
- [11] Omnichord - <https://it.wikipedia.org/wiki/Omnichord>
- [12] AUMI Software - <http://aumiapp.com/about.php>
- [13] KAiKu Music Gloves - <https://www.kaikumusicglove.com/>
- [14] Microsoft Kinect https://it.wikipedia.org/wiki/Microsoft_Kinect
- [15] Real-Time Musical Conducting Gesture Recognition Based on a Dynamic Time Warping Classifier Using a Single-Depth Camera - Fahn Chin-Shyurng, Shih-En Lee and Meng-Luen Wu, Department of Computer Science and Information Engineering, National Taiwan University of Science and Technology, Taipei 10607, Taiwan Dalian University of Technology
- [16] AeroMidi - <https://aeromidi.net/index.php>
- [17] Hand Gesture Recognition with Leap Motion - Youchen Du, Shenglan Liu, Lin Feng, Menghui Chen, Jie Wu
- [18] Virtual Piano - <https://gallery.leapmotion.com/virtual-piano-for-beginners/>
- [19] Natural User Interface - Next Mainstream Product User Interface, WeiyuanLiu, Guilin University Of Electronic Technology, School Of Art&Design Industiral Design, Guilin China
- [20] Leap Motion Controller by UltraLeap - https://www.ultraleap.com/datasheets/Leap_Motion_Controller_Datasheet.pdf
- [21] Foto margine di errore Frequenze di campionamento <https://www.headphonesty.com/2019/12/dacs-guide-part-1/>
- [22] Wikipedia Feature Scaling - https://en.wikipedia.org/wiki/Feature_scaling
- [23] Kinect figure <https://creativevreality.wordpress.com/2016/01/26/setting-up-the-kinect-on-osx-el-capitan/>

BIBLIOGRAFIA

- [24] Wiimote figure - https://en.wikipedia.org/wiki/Wii_Remote/
- [25] Unity Documentation - <https://docs.unity3d.com/Manual>
- [26] Script lifecycle overview - <https://docs.unity3d.com/Manual/ExecutionOrder.html>
- [27] Fondamenti di Acustica - <https://it.wikipedia.org/wiki/Suono>
- [28] Audio Digitale - https://it.wikipedia.org/wiki/Audio_digitale
- [29] C# Synth Project - <https://archive.codeplex.com/?p=csharpsynthproject>
- [30] Mixing digital audio - <http://www.vttoth.com/CMS/index.php/technical-notes/68>
- [31] Sintesi Additiva - [https://it.wikipedia.org/wiki/Sintesi_additiva_\(musica_elettronica\)](https://it.wikipedia.org/wiki/Sintesi_additiva_(musica_elettronica))
- [32] Unity3D-Coding-Examples by Konstantinos Sfikas -
<https://github.com/konsfik/Unity3D-Coding-Examples>
- [33] Sintesi FM - https://it.wikipedia.org/wiki/Sintesi_FM
- [34] Modulazione AM - https://en.wikipedia.org/wiki/Amplitude_modulation
- [35] VJUI by Keijiro Takahashi - <https://github.com/keijiro/VJUI>